

REST paradigm



Il modello architetturale REST

- Il termine **REST** è l'acronimo di **REpresentational State Transfer**; si riferisce ad un particolare modello di architettura software per i sistemi distribuiti, come ad esempio il World Wide Web.
- Il termine “representational state transfer” è stato introdotto ed utilizzato per la prima volta da **Roy Fielding** all'interno della sua tesi di dottorato nell'anno 2000. Fielding è tra i principali autori del protocollo HTTP.
- Il modello REST è poi stato sviluppato dal **W3C (World Wide Web Consortium)** parallelamente alla specifica **HTTP 1.1**, non a caso il World Wide Web rappresenta la più grande implementazione di sistemi **RESTful** (cioè conformi al modello architetturale REST).



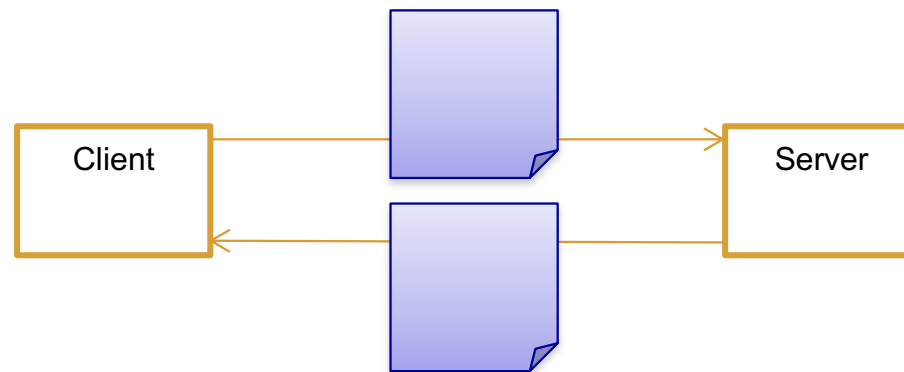
Il modello architetturale REST

Il modello REST è basato sul concetto di client-server. Esiste infatti un client che effettua una richiesta ad un server, il server processa la richiesta e restituisce al client una risposta.

Richiesta e risposta si basano sul concetto di **trasferimento** dello **stato** di una **risorsa**, attraverso una sua **rappresentazione**.

Dobbiamo intendere una risorsa come un qualcosa di tangibile nel mondo reale (un utente, un oggetto) la cui rappresentazione è costituita da un documento che ne rappresenta il suo stato attuale o quello futuro (una foto ad esempio).

Il client effettua il trasferimento dello stato di una risorsa ad un server per richiedere a quest'ultimo di variane lo stato (scrittura), oppure più semplicemente richiede al server che gli venga restituito lo stato attuale di una determinata risorsa (lettura).



Obiettivi del modello REST

L'obiettivo del modello architetturale REST è quello di apportare numerosi benefici ai sistemi che lo implementano. REST infatti intende emulare il comportamento dell'applicazione ottimale.

Tra i benefici indotti dall'implementazione del modello REST citiamo i seguenti:

- **Performance:** applicazioni più performanti perchè meno complesse e ben organizzate
- **Scalability:** sistemi scalabili perchè fortemente disaccoppiati
- **Simplicity:** applicazioni più semplici perchè si occupano di pochi compiti ben definiti
- **Modifiability:** architetture duttili e malleabili proprio perchè “loosely coupled”
- **Visibility:** ogni attore ha visibilità limitata agli altri attori con cui interagisce
- **Portability:** client e server sono “portabili” proprio perchè implementano uno standard ben definito
- **Reliability:** si tratta di un side-effect dovuto ai benefici descritti precedentemente

Vincoli imposti dal modello REST

L'adozione del modello REST presuppone il rispetto di alcuni vincoli architetturali, ognuno dei quali, se adottato, apporta specifici benefici che sono quelli previsti dal modello stesso:

Client-server: L'architettura dev'essere di tipo client-server. Il beneficio che ne deriva sta nella separazione di responsabilità tra client e server (principio **“separation of concern”**).

Ad esempio il client non dovrà preoccuparsi della memorizzazione delle risorse (che risiedono esclusivamente lato server) cosicché la “portabilità” del client rimanga elevata (il client potrebbe esser sostituito con un'altro che rispetta le medesime specifiche architetturali REST).

Vincoli imposti dal modello REST

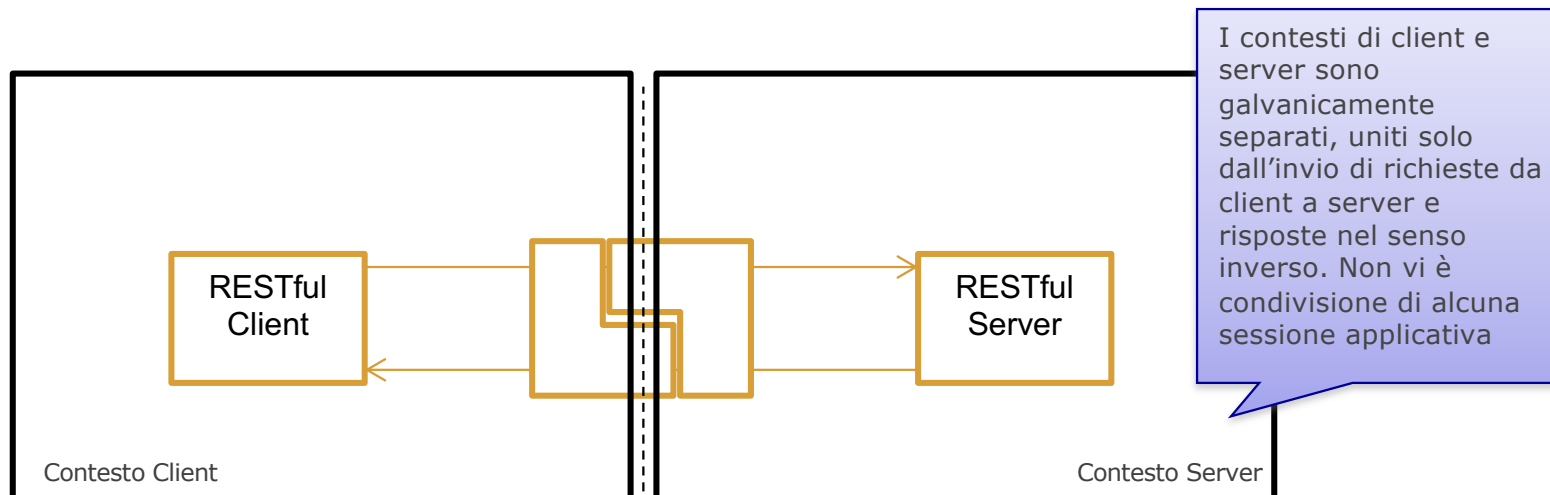
A sua volta il server non deve preoccuparsi della UI (user interface) così da essere enormemente semplificato e scalabile. Sia il server che il client possono essere sostituiti e sviluppati indipendentemente sino a quando le specifiche dell'interfaccia rimangono le medesime e condivise.



Vincoli imposti dal modello REST

- **Stateless:** La comunicazione client-server dev'essere di tipo stateless, vale a dire che nessuna informazione relativa alla "conversazione" dev'essere mantenuta sul server.

Ogni richiesta che il client effettua al server deve riportare tutte le informazioni necessarie a quest'ultimo per l'elaborazione, quindi senza l'utilizzo di una sessione applicativa condivisa tra i due.

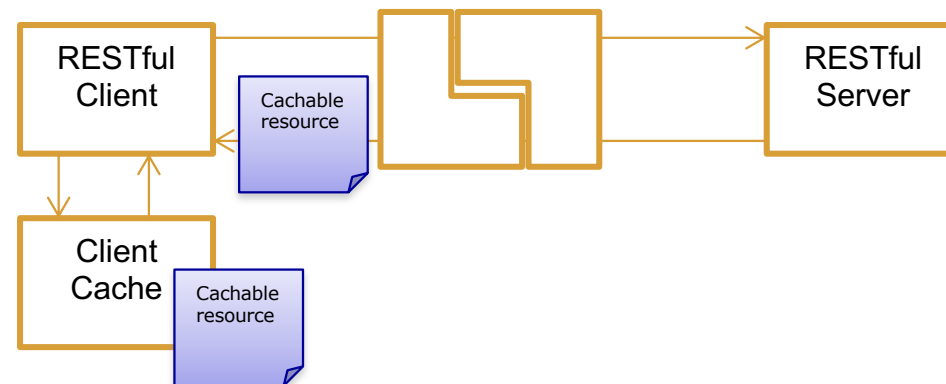


Vincoli imposti dal modello REST

- **Caching:** Il client, eventualmente, può effettuare il caching delle risorse ricevute dal server. Il server, quindi, dovrà specificare se la risorsa erogata al client può esser cachata da quest'ultimo oppure no (questo per evitare che il client utilizzi una risorsa con uno stato non consistente).

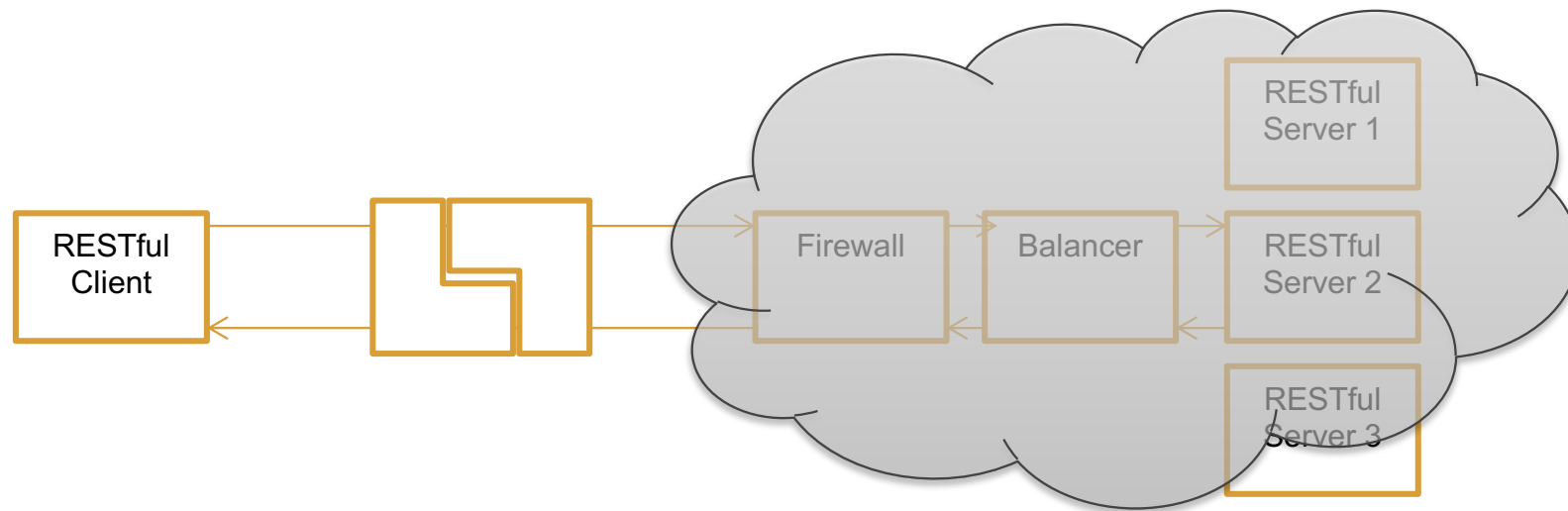
Un utilizzo ottimale del meccanismo di caching può eliminare gran parte delle interazioni client-server, aumentando le performance globali del sistema così come la scalabilità.

NB: Caching non vuol dire storage della risorsa lato client, la responsabilità dello storage è del server



Vincoli imposti dal modello REST

- **Layered (Tiered) System:** Il client può non essere al corrente se è direttamente connesso al server oppure a qualche livello intermedio come firewall, bilanciatori etc. Frapporre ulteriori livelli tra client e server può servire per la gestione della sicurezza, aumentare l'alta affidabilità del server, etc.

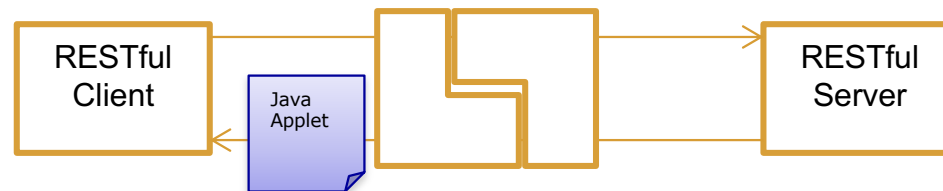


Il modello architetturale REST

Vincoli imposti dal modello REST

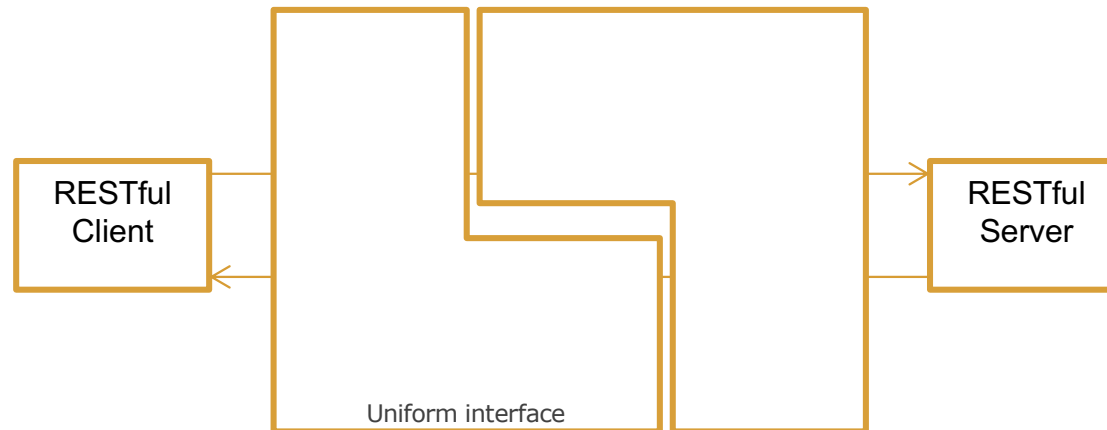
Code on Demand (opzionale): Il server può estendere le funzionalità del client fornendogli del codice che quest'ultimo manderà in esecuzione.

Il classico esempio di Code on Demand è l'utilizzo client-side di Java Applet o Script che il server invia al client e quest'ultimo li esegue.



Vincoli imposti dal modello REST

- **Uniform interface:** La presenza di un'interfaccia uniforme e condivisa tra client e server è un principio (vincolante) fondamentale del modello REST proprio perchè aiuta a disaccoppiare i due, consentendo ad entrambi di evolvere separatamente.



Principi guida nella definizione dell'interfaccia

La definizione dell'interfaccia REST tra client e server è governata dai seguenti principi:

- **Identification of resources:** ogni risorsa dev'essere identificata univocamente (ad esempio tramite un **URI** – **Uniform Resource Identifier**)
- **Manipulation of resources through the representations:** quando il client entra in possesso di una risorsa, esso ha tutte le informazioni necessarie per modificarne lo stato
- **Self-descriptive messages:** ogni messaggio scambiato tra client e server contiene tutte le informazioni necessarie per la sua elaborazione. Ad esempio l'Internet Media Type descrive quale parser utilizzare per trattare il messaggio (XML, JSON, etc), oppure le informazioni di caching di una risorsa istruiscono il client su come effettuarne delle copie locali, etc
- **Hypermedia as the engine of application state (aka HATEOAS):** il client interagisce con il server esclusivamente tramite link (hypermedia) forniti dal server stesso. Un client REST non conosce a priori come interagire con il server, fatta eccezione per la capacità di processare tipi diversi di hypermedia. Questo approccio differisce con altri paradigmi client-server (SOA ad esempio) dove il client interagisce con il server sulla base di un documento d'interfaccia bene definito.
Tale principio disaccoppia ulteriormente il client dal server.

Risorsa

E' una qualsiasi forma tangibile appartenente al mondo reale (un utente, un oggetto).

Possiamo anche pensare ad una risorsa come una sorgente di informazioni relative ad una specifica entità, a cui si può accedere utilizzando un identificativo globale



Identificativo di una risorsa

- Trattasi di un identificativo che distingue quella risorsa univocamente all'interno dell'applicazione.
- Adottando il modello REST in un'applicazione web, l'identificativo di una risorsa è costituito dal suo URI (Uniform Resource Identifier), ad esempio:
- <http://www.sunnyvale.it/intranet/utenti/1>
- Identifica univocamente l'utente con ID = 1



Stato di una risorsa

Il valore che assumono le sue proprietà in un determinato momento.

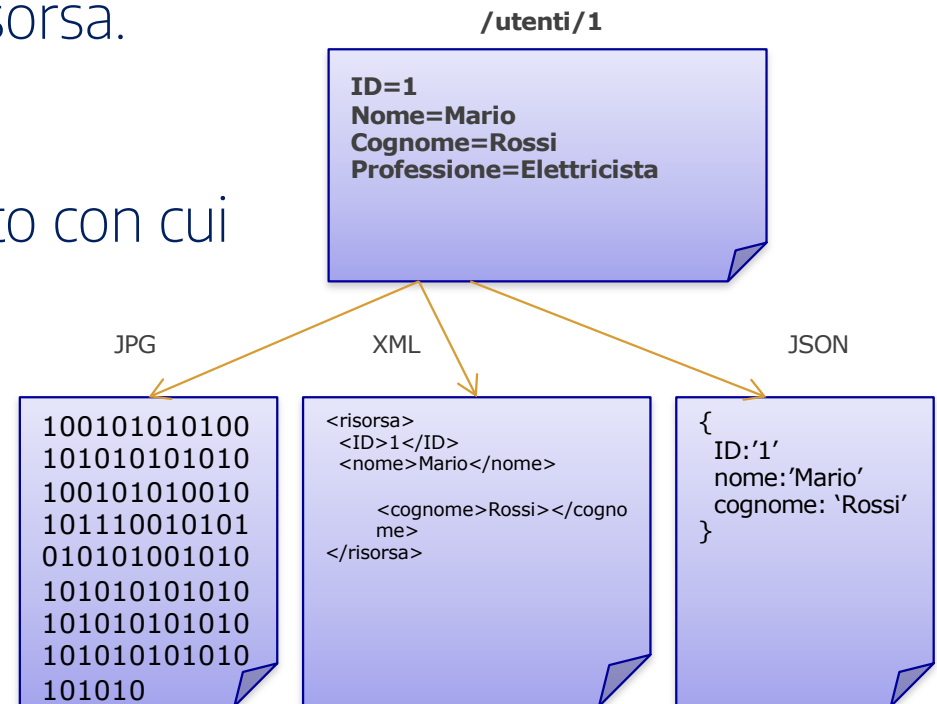
Nella terminologia software possiamo riferirci ad un oggetto di una determinata classe, con proprietà il cui valore è specifico di un preciso momento.

/utenti/1

ID=1
Nome=Mario
Cognome=Rossi
Professione=Elettricista (il
valore di questa proprietà è riferito
ad un determinato momento)

Rappresentazione di una risorsa

- E' il formato scelto dal server (o dal client) per rappresentare una determinata risorsa.
- Client e server negoziano il formato con cui verrà rappresentare la risorsa.



Operazioni (o verbi)

Sono le operazioni che un client può effettuare sulle risorse tramite il server. Alcune di esse non modificano lo stato della risorsa altre lo alterano oppure cancellano la risorsa stessa.

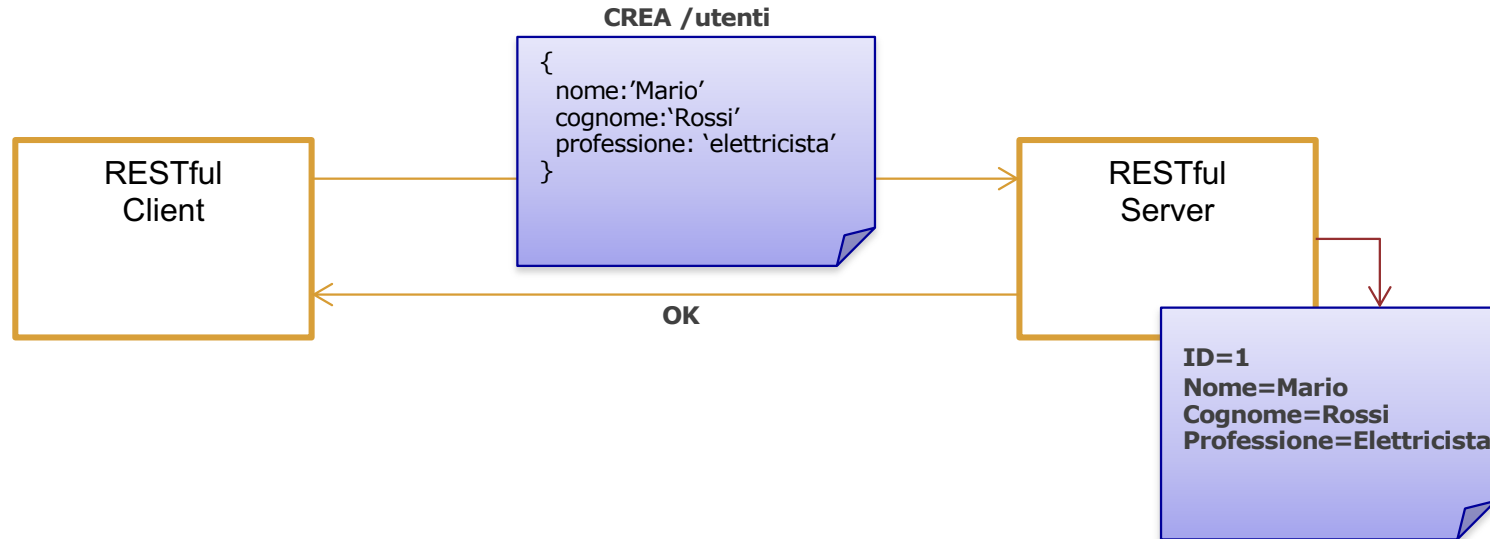
A titolo esemplificativo citiamo le seguenti operazioni:

- CREA
- LEGGI
- MODIFICA
- CANCELLA

Si tratta delle tipiche operazioni CRUD (Create, Read, Update, Delete)

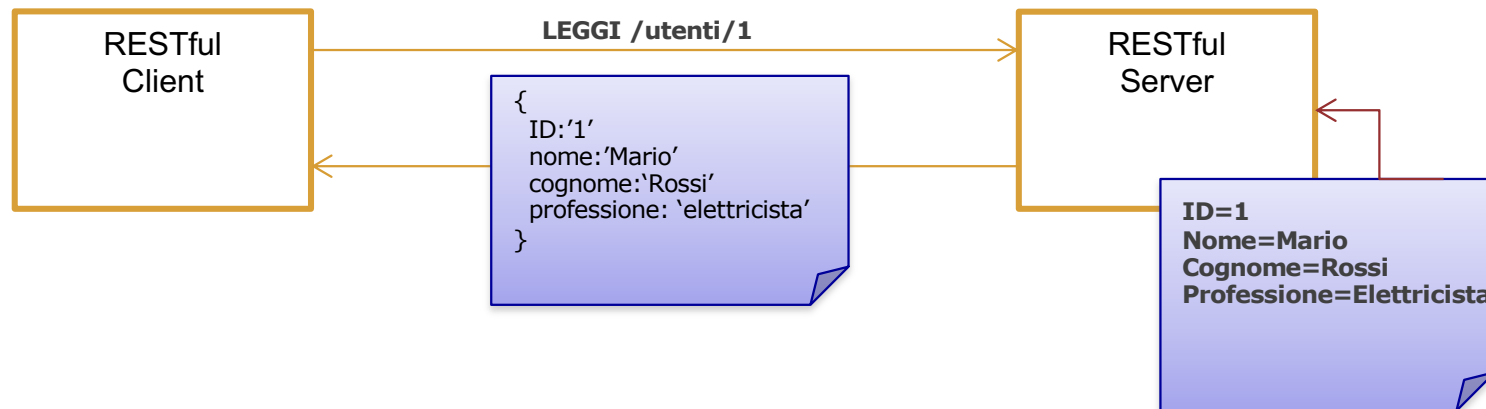
Operazioni: CREA

Nella fase di creazione di una risorsa, il client costruisce una rappresentazione della stessa e la invia al server, il quale salva la risorsa all'interno del suo storage e notifica il client che l'operazione è avvenuta con successo.



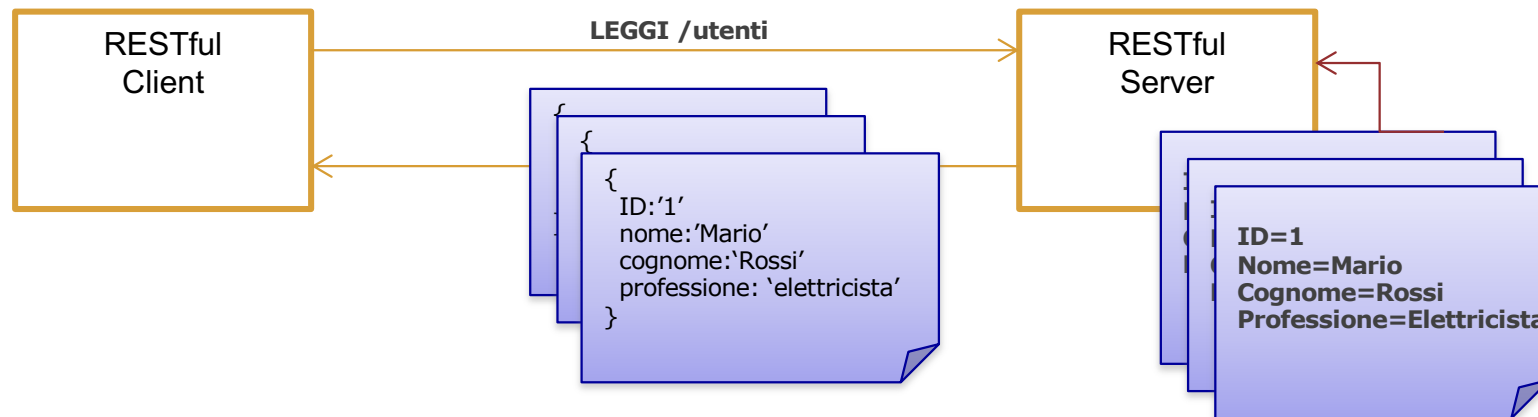
Operazioni: LEGGI

Nella fase di lettura di una risorsa, il client richiede al server la risorsa con un determinato URI (**/utenti/1**), il server legge la risorsa dallo storage e ne invia una rappresentazione al client.



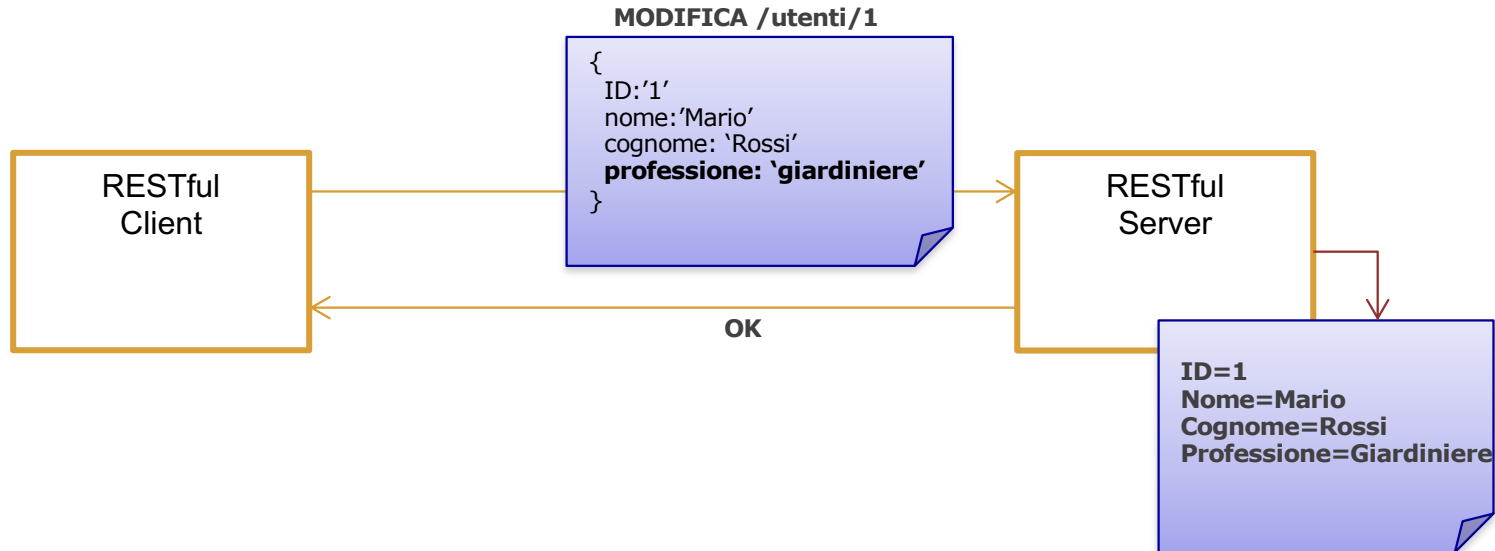
Operazioni: LEGGI (molti)

E' anche possibile richiedere in lettura più di una risorsa riferendosi ad un URI più generico (**/utenti**).



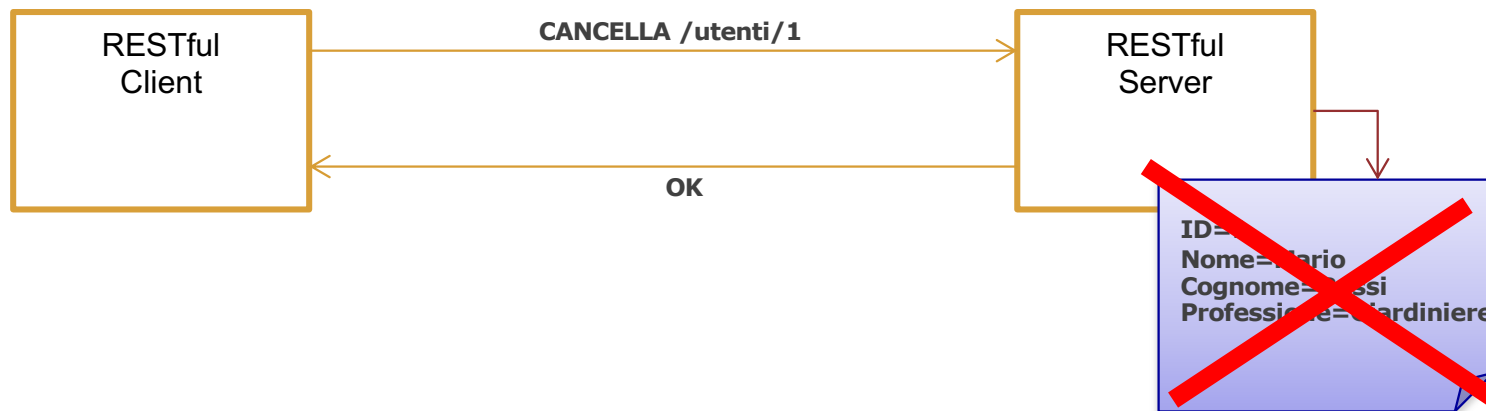
Operazioni: MODIFICA

Nella fase di modifica di una risorsa, il client, dopo averne richiesto una rappresentazione al server, modifica il valore delle sue proprietà e la rispedisce al server che la salverà nel suo storage. Il server notificherà il client dell'avvenuta modifica.



Operazioni: CANCELLA

Per cancellare una risorsa dal server, il client farà richiesta specificando l'URI della risorsa, il server dopo aver compiuto l'operazione restituirà un messaggio al client per notificarlo dell'avvenuta operazione.



Operazioni

- Benchè le operazioni possibili sulle risorse possano essere molteplici, nelle slide precedenti abbiamo descritto le più comuni.
- A volte ci si riferisce alle operazioni con il termine “verbi”, questo perché è facile immaginare il client che tramite un verbo richiede una specifica operazione da effettuarsi su di una risorsa identificata dal suo URI (**CANCELLA /utenti/1**, **LEGGI /utenti/1**, etc).

Riassumendo...

Un concetto importante in REST è l'esistenza di risorse (fonti di informazioni), a cui si può accedere tramite un identificatore globale (un URI). Per utilizzare le risorse, le componenti di una rete (componenti client e server) comunicano attraverso una interfaccia standard (ad es. HTTP) e si scambiano rappresentazioni di queste risorse (il documento che trasmette le informazioni).

Un numero qualsiasi di componenti (client, server, cache, tunnel ecc.) può mediare la richiesta, ma ogni componente interviene senza conoscere lo storico di esse (stateless)

Di conseguenza un'applicazione può interagire con una risorsa conoscendo due cose: l'identificatore della risorsa e l'azione richiesta - non ha bisogno di sapere se ci sono proxy, gateway, firewalls, tunnel, etc tra essa ed il server su cui è presente l'informazione cercata.

Riassumendo...

- L'applicazione comunque deve conoscere il formato dell'informazione (rappresentazione) restituita, tipicamente un documento HTML, XML o JSON, ma potrebbe essere anche un'immagine o qualsiasi altro contenuto.
- Per saperne di più
http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm,
link al capitolo 5 della tesi di dottorato di Roy Fielding dove si introduce il concetto di REST.

REST nel mondo reale (Non solo HTTP)

Sebbene sia nato e cresciuto con la tecnologia HTTP, il modello architetturale REST ed i concetti da lui introdotti sono stati implementati in numerosi sistemi software distribuiti.

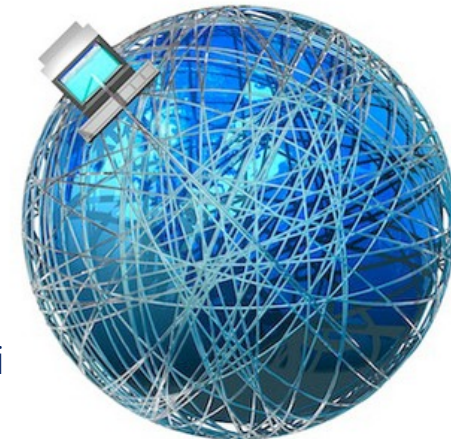
- **MySQL Cluster** è un database le cui funzionalità vengono esposte secondo un paradigma REST da uno specifico modulo di Apache Web Server.
- L'**Atom Publishing Protocol**, il protocollo utilizzato per la pubblicazione dei blog, è considerato un'implementazione del modello REST
- Numerosi **siti internet** vengono esposti all'utente con un tipico modello REST (le risorse sono le pagine)
- **Sun Microsystems' Cloud API**, delle API per la gestione di risorse in un Cloud, sono implementate secondo il modello REST
- **Google Fusion Tables**, un strumento di Google per la condivisione dati e collaborazione è richiamabile tramite Web Services che utilizzano il paradigma REST.

RESTful Web Services



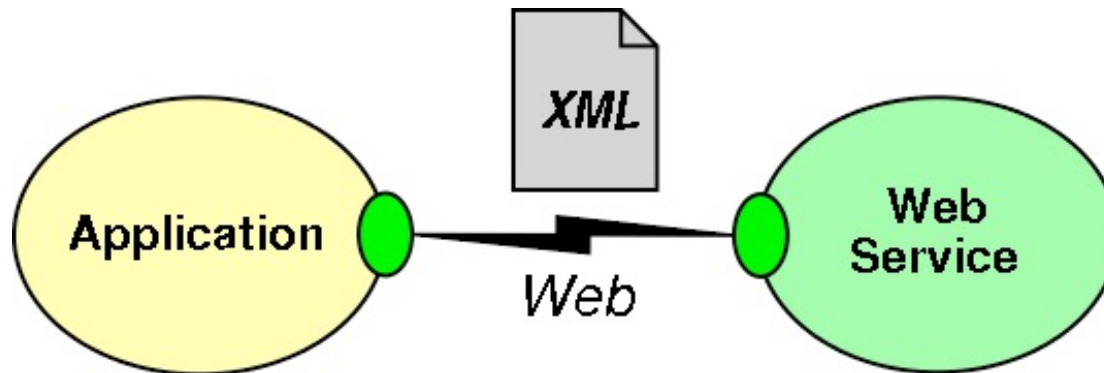
RESTful Web Services

- Il **Web** è nato negli anni '90 come una piattaforma per la condivisione di documenti distribuiti su diverse macchine e tra loro interconnessi. Il tutto è nato e si è evoluto grazie alla standardizzazione di alcuni semplici concetti:
- **URL/URI** – meccanismo per individuare risorse in una rete
- **HTTP** – protocollo semplice e leggero per richiedere una risorsa ad una macchina
- **HTML** – linguaggio per la rappresentazione dei contenuti
- Questa semplice idea iniziale si è evoluta nel corso degli anni non tanto nei concetti di base, quanto nel modo di intenderli e di utilizzarli.
- Al testo si sono aggiunti contenuti multimediali, i documenti sono generati dinamicamente e non pubblicati come pagine statiche, e poi **il Web** esce dalla visione esclusivamente ipertestuale per diventare un contenitore di applicazioni software interoperabili: **una piattaforma applicativa distribuita (i Web Services)**.



Dal Web ai Web Services

- Intorno all'anno 2000 ha origine il concetto di **Web Service**: un sistema software progettato per supportare un'interazione tra applicazioni, utilizzando le tecnologie e gli standard Web.
- Il meccanismo dei **Web Service** consente di far interagire in maniera trasparente applicazioni sviluppate con linguaggi di programmazione diversi, in esecuzione su sistemi operativi eterogenei.
- Questo meccanismo consente di realizzare porzioni di funzionalità in maniera indipendente e su piattaforme potenzialmente incompatibili facendo interagire i vari pezzi tramite tecnologie Web e creando un'architettura facilmente componibile (SOA).



Due tipi di Web Services

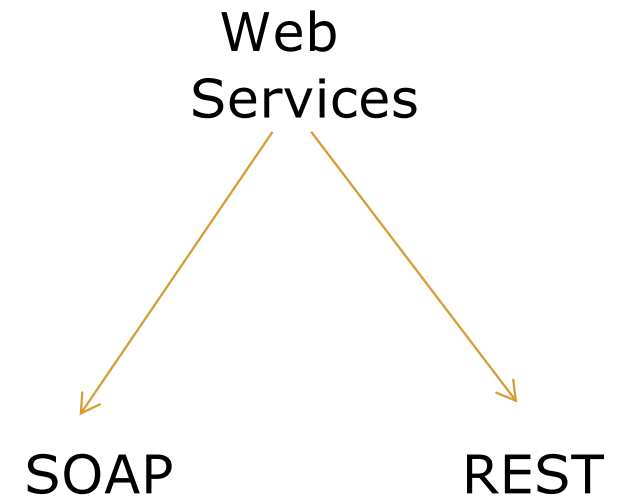


Esistono due approcci alla creazione di Web Service:

- Un approccio è basato sul protocollo standard **SOAP** (Simple Object Access Protocol), per lo scambio di messaggi per l'invocazione di servizi remoti, si prefigge di riprodurre in ambito Web un approccio a chiamate remote, *Remote Procedure Call*, tipico di protocolli di interoperabilità come CORBA, DCOM e RMI.
- Un secondo approccio è ispirato ai principi architetturali tipici del Web e si concentra sulla descrizione di risorse, sul modo di individuarle nel Web e sul modo di trasferirle da una macchina all'altra. Questo è l'approccio che si ispira al modello architetturale **REST (REpresentational State Transfer)**.



In questo capitolo vedremo in cosa consiste l'approccio **REST** per la realizzazione di Web Service, vedremo come rappresentare risorse e come gestire il loro stato, e faremo frequenti paragoni con l'altro tipo di implementazione.



Concetti REST alle architetture Web Services

Concetto REST	Adozione in RESTful Web Services	Corrispettiva implementazione nei Web Services SOAP
Risorsa	Entità messa a disposizione dall'HTTP server	XML element
Identificazione risorsa	Uniform Resource Identifier (URI)	Address location + SOAPAction HTTP Header
Rappresentazione risorsa	Internet Media Type (MIME types)	Document Literal / RPC Encoded / MIME attachment
Risorse autodescrittive	Accept e Content-type HTTP headers	XML Schema / WSDL
Operazioni sulle risorse	Metodi HTTP (POST, GET, PUT, DELETE)	WS operations
Notifica risultato operazioni	HTTP Return codes (200, 204, 401, 404, 500, 503)	SOAP Response / SOAP Fault

Concetti REST alle architetture Web Services

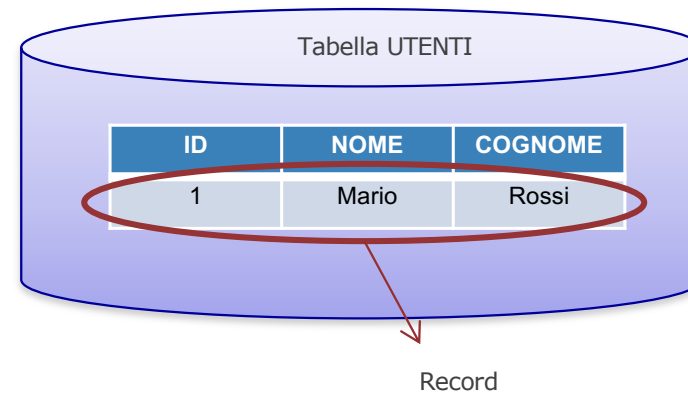
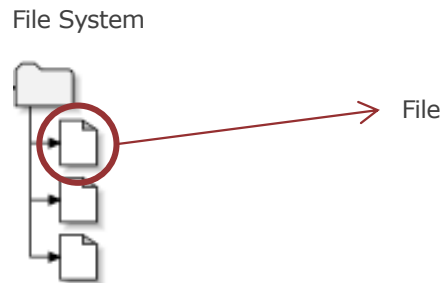
Concetto REST	Adozione in RESTful Web Services	Corrispettiva implementazione nei Web Services SOAP
No client context being stored on the server	Comunicazione HTTP Stateless	Comunicazione HTTP Stateless
Caching delle risorse	HTTP Caching	NO Caching
HATEOAS	Hyperlinks	NO Hyperlinks, Entry point multipli
Sicurezza	HTTP Security mechanisms	WS Security / XML Signature

Perchè REST è meglio

- Tutto il necessario per invocare procedure remote su di un server è già stato previsto dal protocollo HTTP e non vi è la necessità di protocolli on-top (SOAP)
- Il protocollo SOAP è estremamente verboso e ridondante rispetto al modello REST
- Vi sono ripercussioni sulla sicurezza delle comunicazioni, occorre implementare firewall che comprendano i meccanismi di sicurezza e cifratura XML/SOAP

Esempio di RESTful Web Service

- Una risorsa è un'entità lato server che il WS RESTful mette a disposizione al client



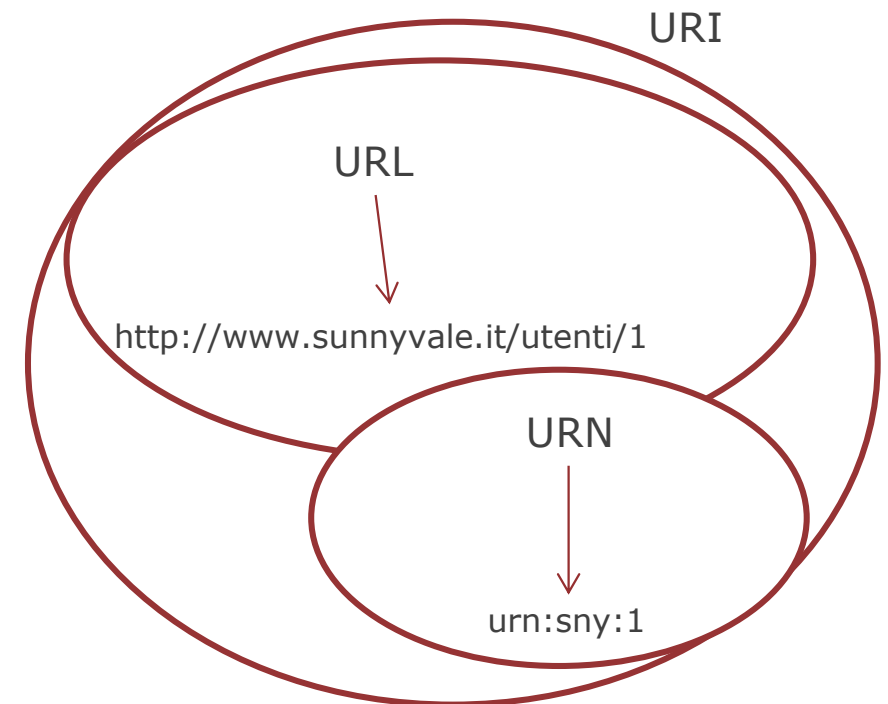
Identificazione delle risorse

Nei RESTful Web Service ogni risorsa è identificata con un suo **URI (Uniform Resource Identifier)**.

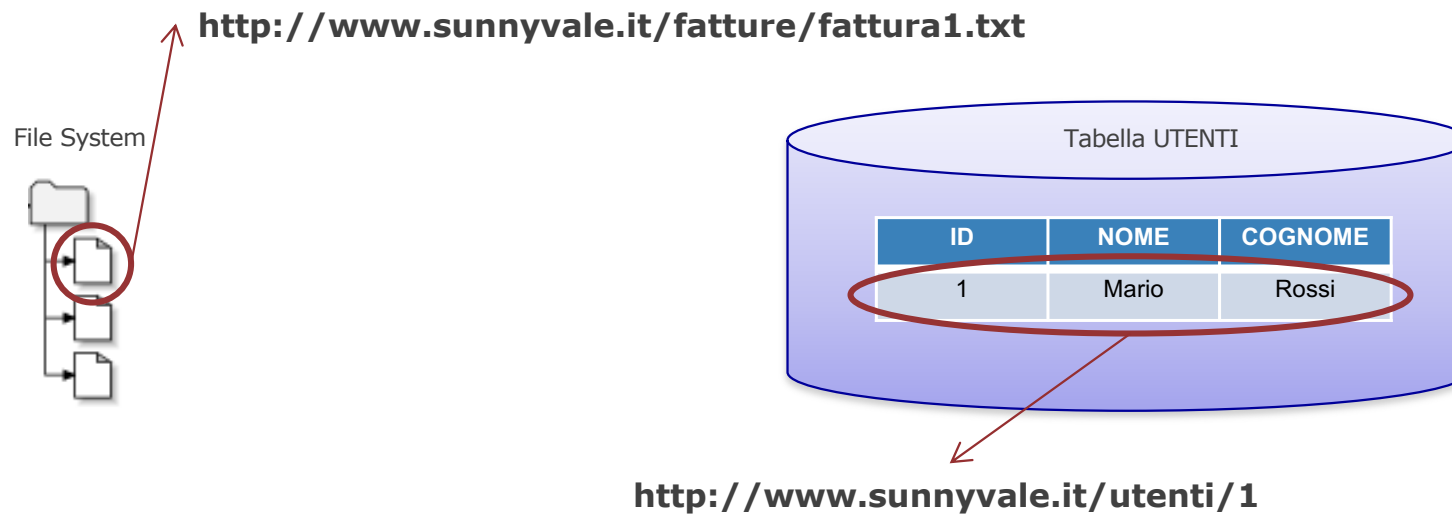
Un Uniform Resource Identifier (URI, acronimo più generico rispetto ad "URL") è una stringa che identifica univocamente una risorsa generica che può essere un indirizzo Web, un documento, un'immagine, un file, un servizio, un indirizzo di posta elettronica, ecc. L'URL è un URI, o più comunemente chiamato indirizzo web.

Identificazione delle risorse

- Un **URL (Uniform Resource Locator)** è un URI che, oltre a identificare una risorsa, fornisce mezzi per agire su o per ottenere una rappresentazione della risorsa descrivendo il suo meccanismo di accesso primario o la sua "ubicazione" ("location") in una rete.
- Un **URN (Uniform Resource Name)** è un URI che identifica una risorsa mediante un "nome" in un particolare dominio di nomi ("namespace"). Un URN può essere usato per parlare di una risorsa senza lasciar intendere la sua ubicazione o come ottenerne una rappresentazione. Per esempio l'URN `urn:isbn:0-395-36341-1` è un URI che consente di individuare univocamente un libro mediante il suo nome 0-395-36341-1 nel namespace dei codici ISBN, ma non suggerisce dove e come possiamo ottenere una copia di tale libro.

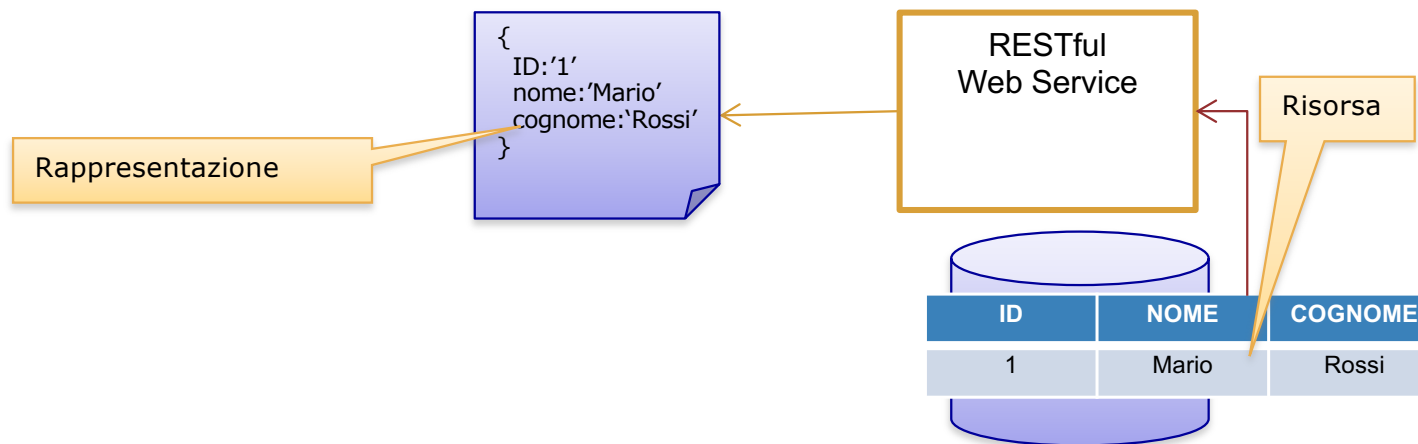


Esempio identificazione risorse



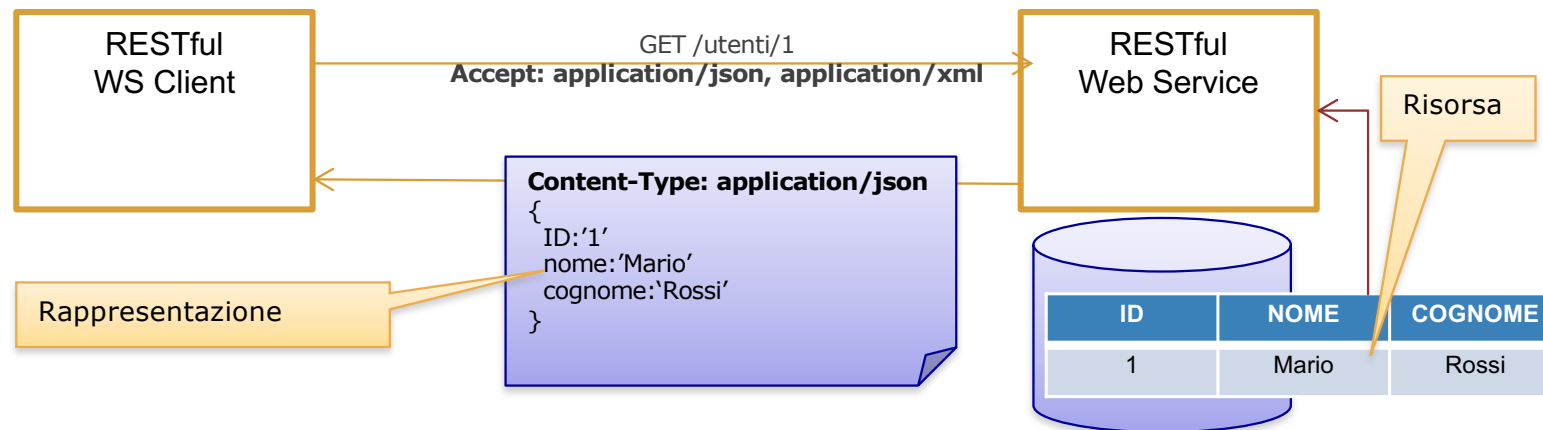
Rappresentazione delle risorse

- Le risorse di per sè sono concettualmente separate dalle rappresentazioni restituite al client. Ad esempio, un Web Service non invia al client direttamente un record del suo database, ma una sua rappresentazione in una codifica dipendente dalla richiesta del client e/o dall'implementazione del servizio.



Risorse autodescrittive

- Il tipo di rappresentazione inviata dal Web Service al client è indicato nella stessa risposta HTTP tramite un tipo MIME, così come avviene nella classica comunicazione tra Web server e browser.
- Un client, a sua volta ha la possibilità di richiedere una risorsa in uno specifico formato sfruttando l'attributo **Accept** di una richiesta HTTP di tipo GET:



Risorse autodescrittive: MIME types

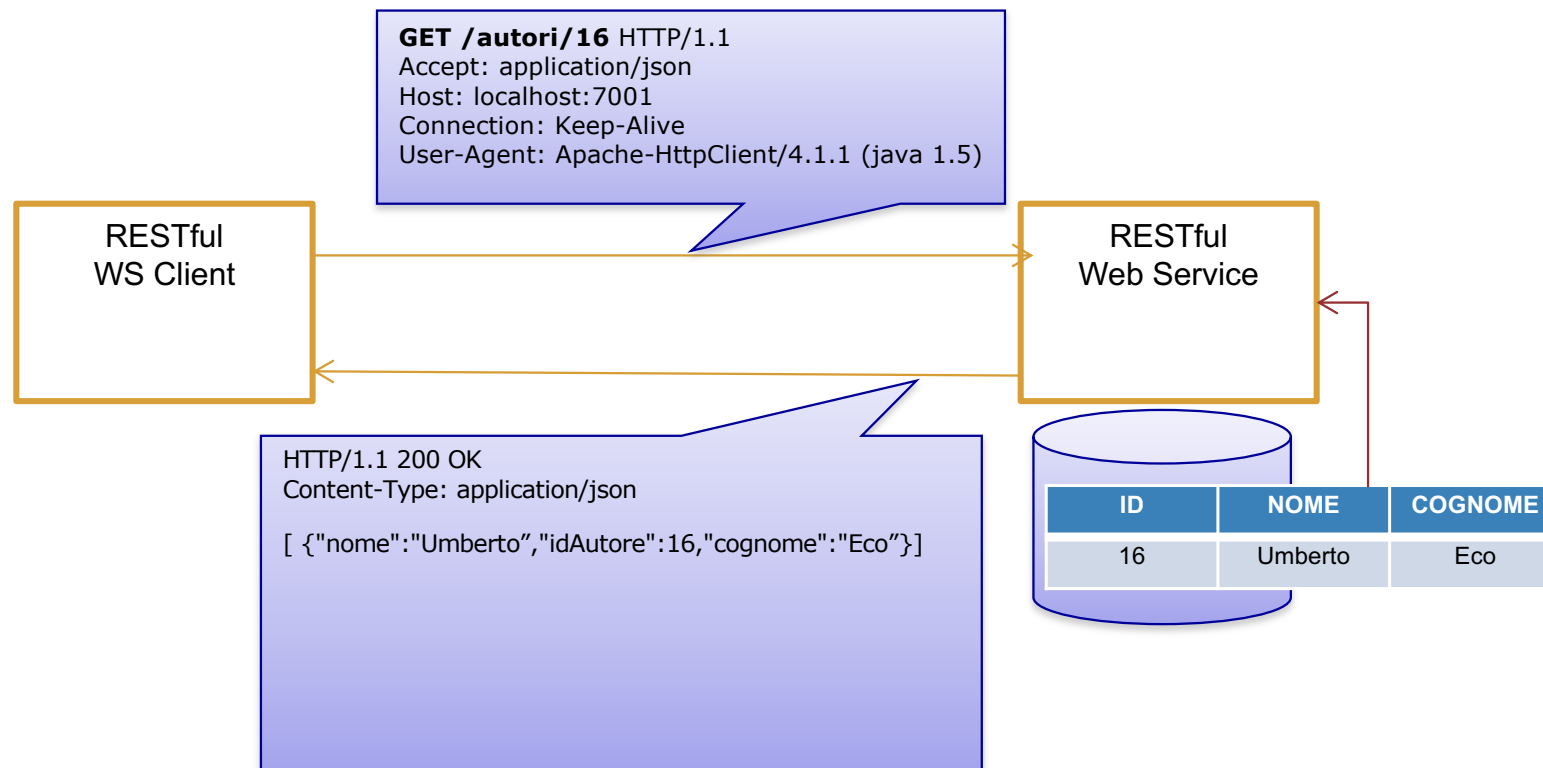
- application/json: JavaScript Object Notation JSON - (RFC 4627)
- application/javascript: ECMAScript/JavaScript - (RFC 4329)
- application/octet-stream: Arbitrary binary data
- application/pdf: Portable Document Format, PDF - (RFC 3778)
- application/xml: XML files - (RFC 3023)
- application/zip: ZIP archive files
- text/html: HTML - (RFC 2854)
- application/vnd.ms-powerpoint Microsoft Powerpoint files

Operazioni sulle risorse

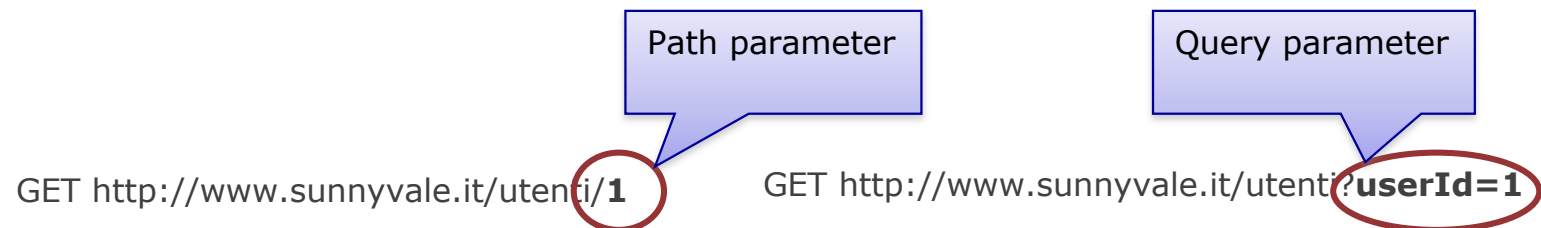
Una volta spiegato come individuare una risorsa abbiamo bisogno di un meccanismo per indicare quali operazioni effettuare su di esse. Il principio dell'**uso esplicito dei metodi HTTP** ci indica di sfruttare i metodi (o verbi) predefiniti di questo protocollo, e cioè **GET**, **POST**, **PUT** e **DELETE**.

METODO HTTP	OPERAZIONE	
GET	Ricerca/Lettura	→ Nullipotent (sola lettura)
POST	Creazione	
PUT	Modifica/Creazione	} Idempotent (modificano la risorsa sul server)
DELETE	Cancellazione	

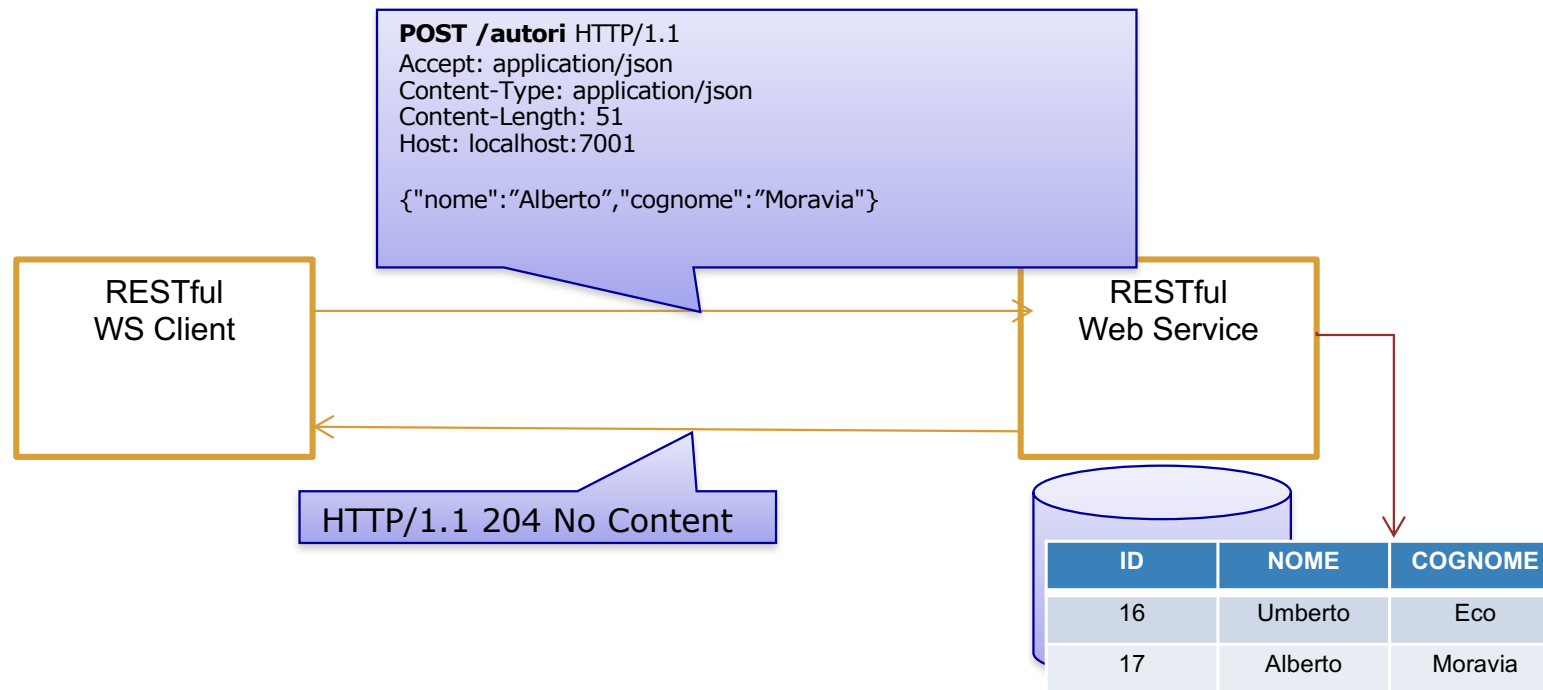
Operazioni sulle risorse: GET (Lettura singola)



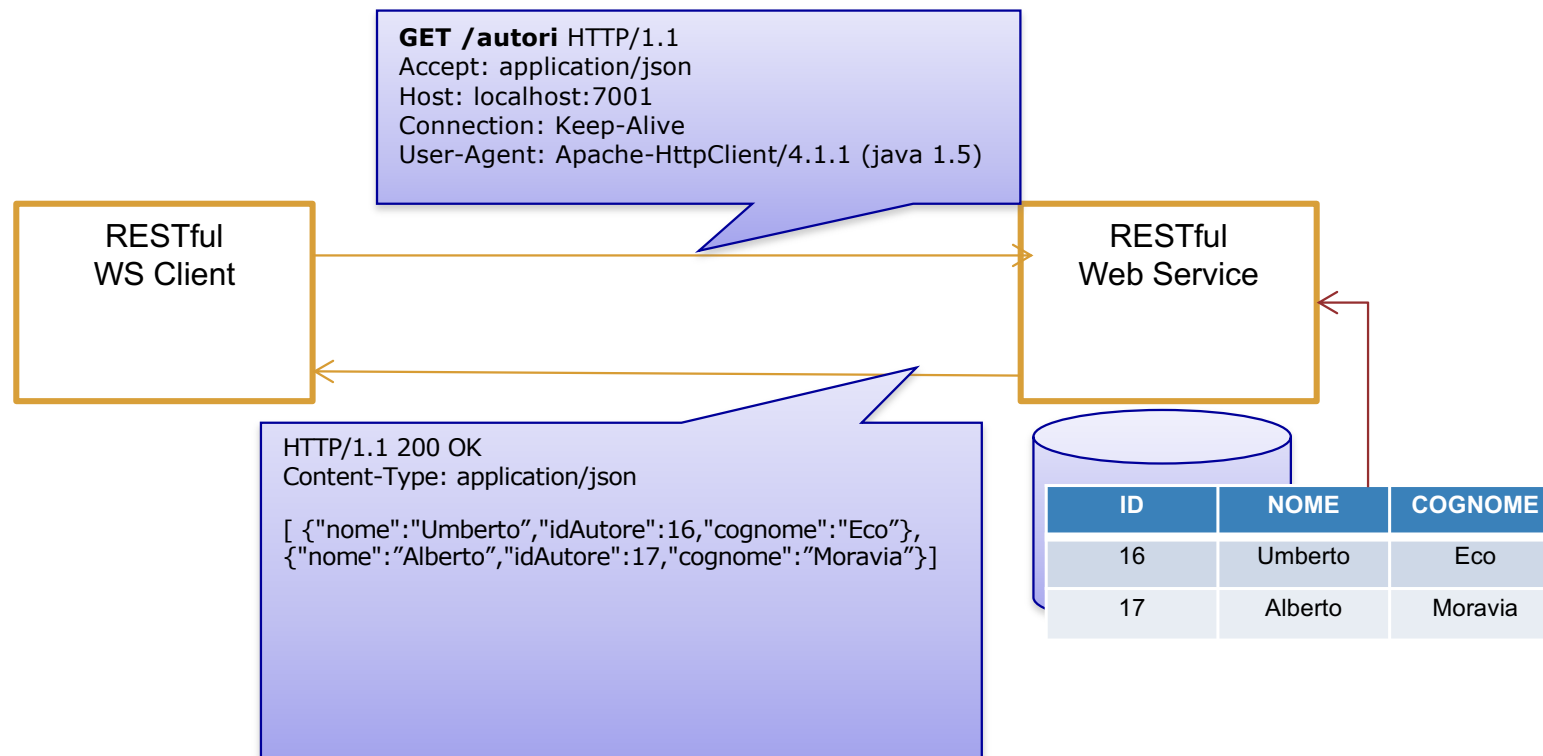
Parametri delle richieste



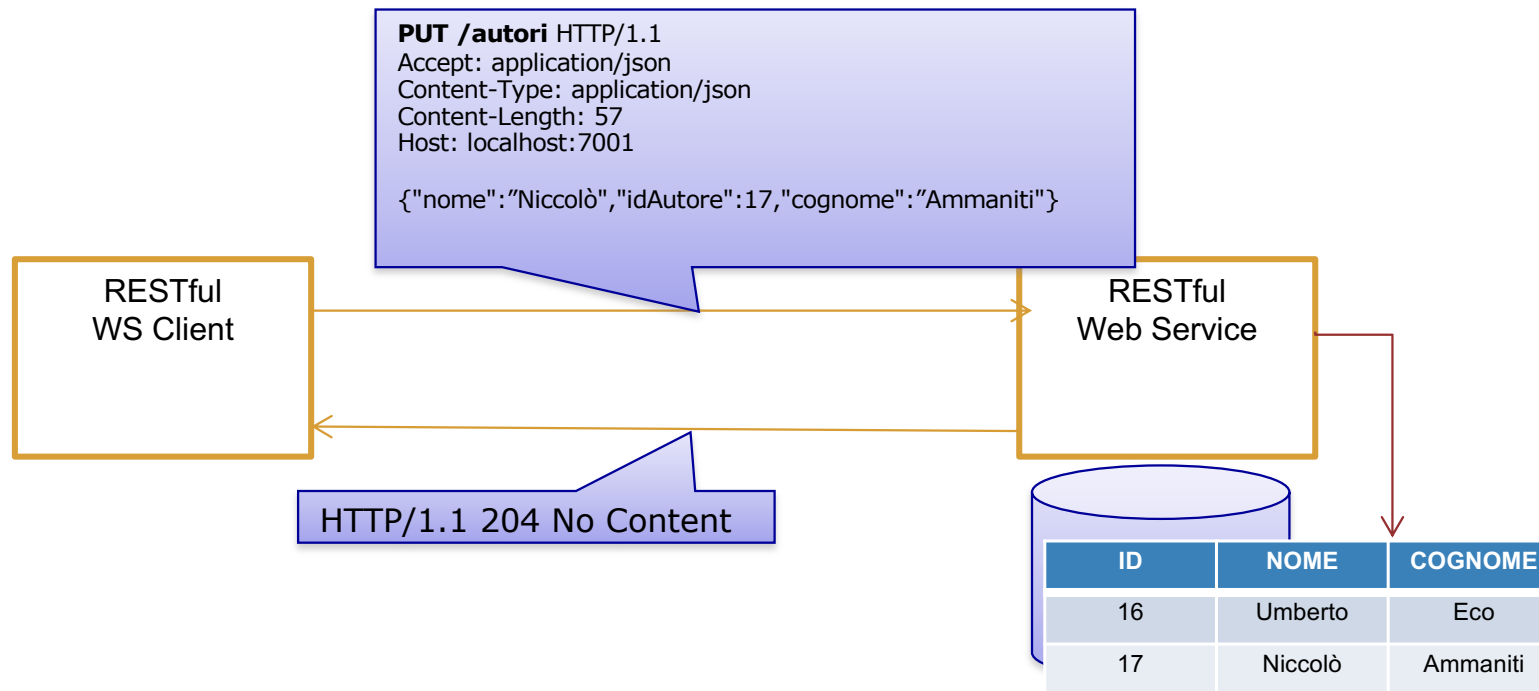
Operazioni sulle risorse: POST (Creazione)



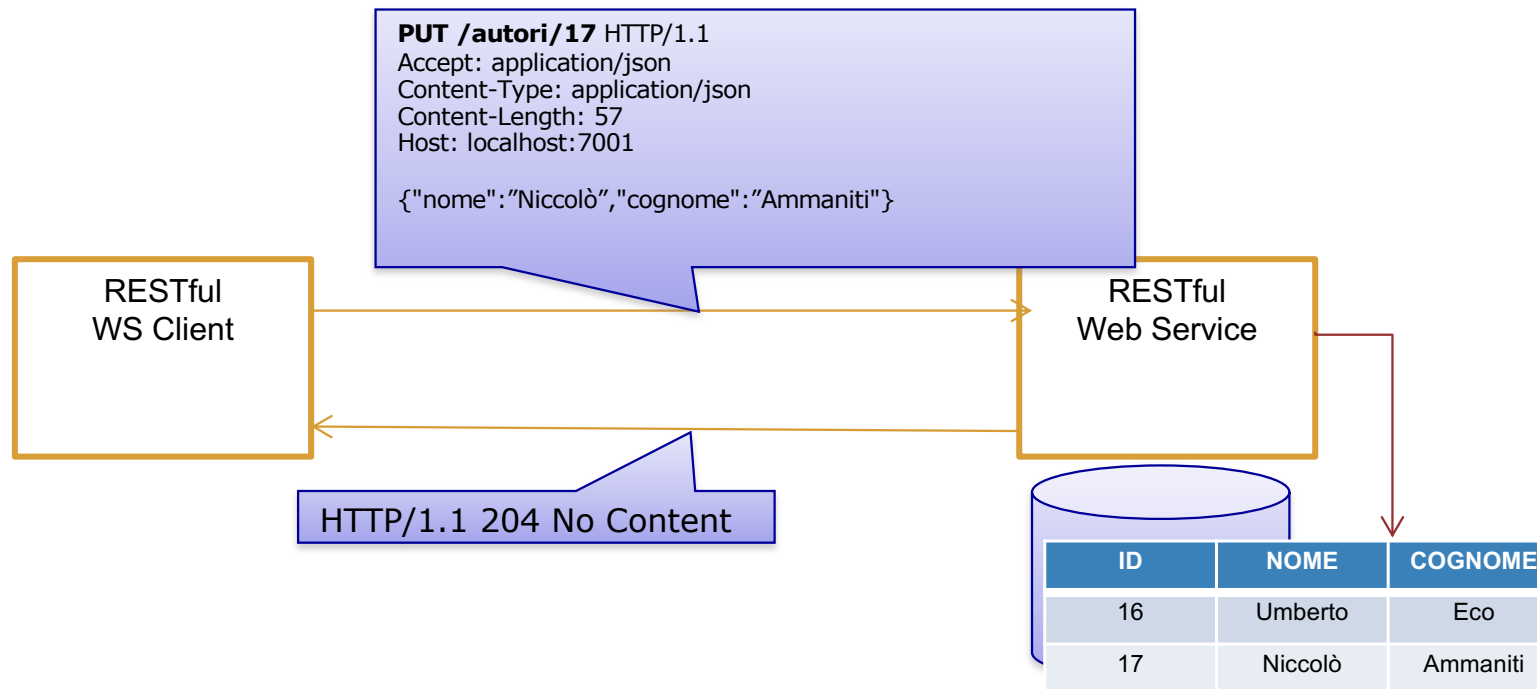
Operazioni sulle risorse: GET (Lettura multipla)



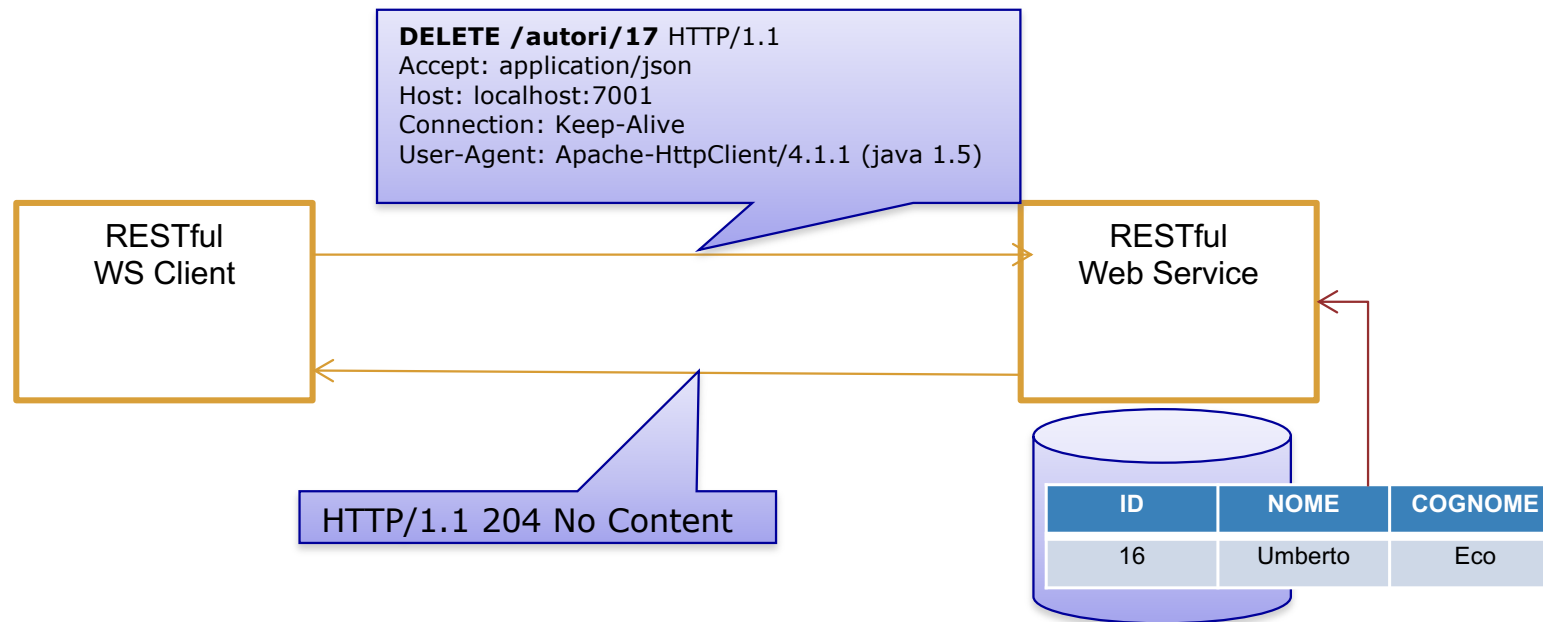
Operazioni sulle risorse: PUT (Modifica/Creazione)



Operazioni sulle risorse: PUT (Modifica/Creazione)



Operazioni sulle risorse: DELETE (Cancellazione)



Notifica risultato operazioni

I Web Services di tipo RESTful utilizzano lo status code di ritorno previsto dal protocollo HTTP per restituire al client il risultato dell'operazione richiesta (oltre a restituire eventualmente una rappresentazione della/e risorsa/e qualora l'operazione fosse di tipo GET).

- 2XX (Success)
 - 200 OK
 - 204 No Content
- 3XX (Redirezione)
 - 303 See Other (da HTTP/1.1)
 - 304 Not Modified
- 4XX (Client Error)
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 405 Method Not Allowed
 - 418 I'm a teapot (pesce d'aprile, esiste davvero)
- 5XX (Server Error)
 - 500 Internal Server Error
 - 503 Service Unavailable
 - 504 Gateway Timeout

No client context stored on the server (Stateless)

Il principio della **comunicazione stateless** è ben noto a chi lavora con il Web. Questa è infatti una delle caratteristiche principali del protocollo HTTP, cioè ciascuna richiesta non ha alcuna relazione con le richieste precedenti e successive. Lo stesso principio si applica ad un Web Service **RESTful**, cioè le interazioni tra client e server devono essere senza stato.

È importante sottolineare che sebbene REST preveda la **comunicazione stateless**, non vuol dire che un'applicazione non deve avere stato. La responsabilità della gestione dello stato dell'applicazione non deve essere conferita al server, ma rientra nei compiti del client.

La principale ragione di questa scelta è la scalabilità: mantenere lo stato di una sessione ha un costo in termini di risorse sul server e all'aumentare del numero di client tale costo può diventare insostenibile. Inoltre, con una comunicazione senza stato è possibile creare cluster di server che possono rispondere ai client senza vincoli sulla sessione corrente, ottimizzando le prestazioni globali dell'applicazione.

Caching

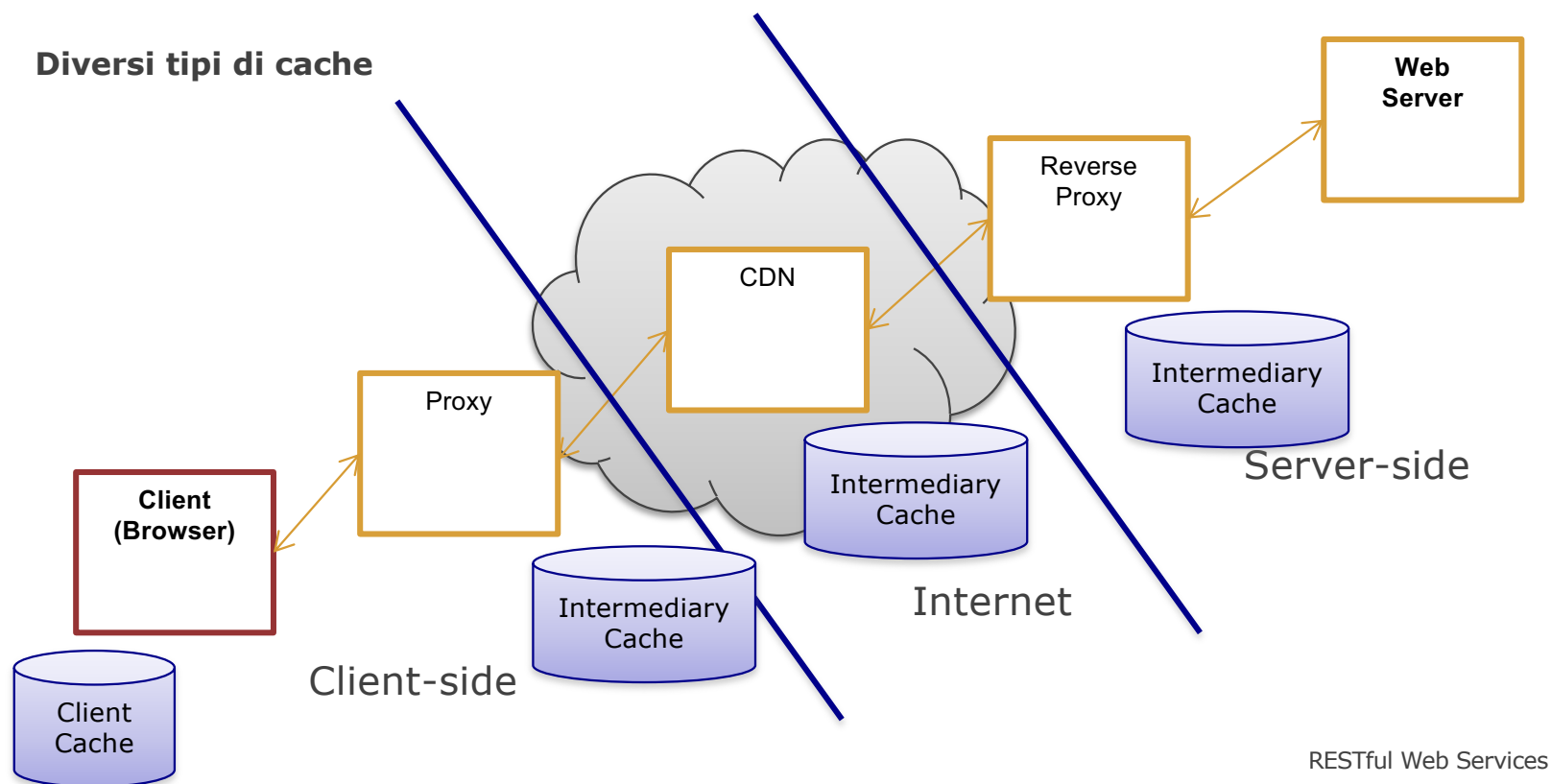
The advantage of adding cache constraints is that they have the potential to partially or completely eliminate some interactions, improving efficiency, scalability, and user-perceived performance by reducing the average latency of a series of interactions.

Roy Fielding Network Cache

Dalle parole di Roy Fielding si evince che il caching delle risorse è importante perchè:

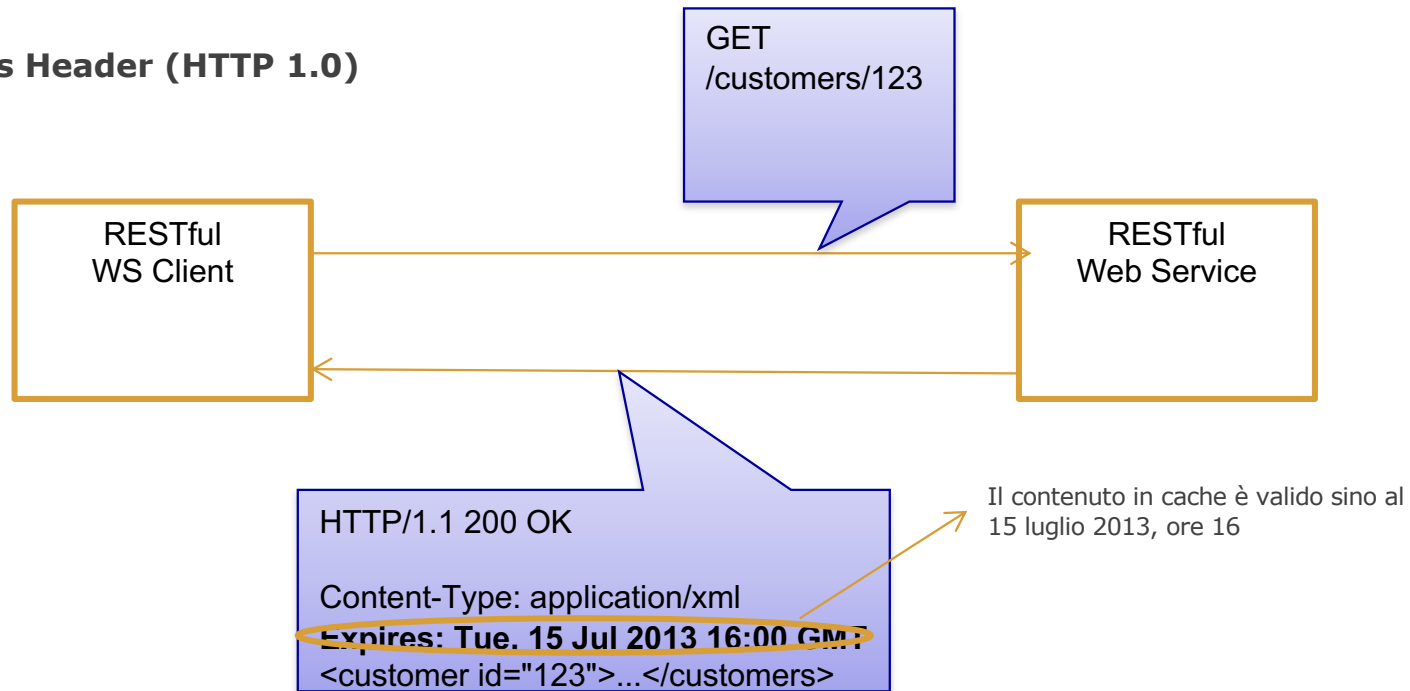
- Aumenta le performance dell'applicazione: i contenuti in cache arrivano prima all'utente rispetto a quelli generati dal server
- Aiuta il fault tolerance: copie di risorse nella cache sono disponibili anche quando il server non è operativo
- Riduce il carico del server: il client utilizzando la copia in cache non effettua chiamate al server
- Aiuta la scalabilità dell'applicazione: Diminuendo il traffico verso il server, molti più client possono collegarsi al sistema

Caching delle risorse: HTTP caching



Caching delle risorse: HTTP caching

Expires Header (HTTP 1.0)



Caching delle risorse: HTTP Caching

Cache-Control Header

- Introdotta con HTTP 1.1
- Maggiori riferimenti per i vari tipi di caching (client, cdn/proxy, gateway)
- Ha reso obsoleto (e deprecato) l'uso dell'header Expires (comunque mantenuto per retrocompatibilità)
- Il valore è composto da un serie di direttive separate da una virgola, ognuna delle quali può esser formata da *chiave= valore*

Caching delle risorse: HTTP Caching

Cache-Control Header Directives

- **private:** indica che nessun intermediario è autorizzato al caching della risorsa (proxy/cdn). Solo il client è autorizzato a fare caching
- **public:** è l'opposto di private, significa che ogni attore che partecipa alla request/response HTTP può fare caching
- **no-cache:** indica che la response non dovrebbe essere cachata, se lo fosse, la risorsa in cache deve venir rivalidata dal server prima di essere utilizzata
- **no-store:** alcune volte il browser sposta la risorsa dalla cache primara (RAM) a quella secondaria (FS) cosicchè possa esser disponibile al reboot della macchina o reload del browser. Con questa direttiva il browser non può effettuare questa operazione
- **no-transform:** alcuni proxy convertono il formato della risorsa (specialmente immagini) per migliorare le performance (ad esempio effettuano la compressione). Con questa direttiva si vieta questa pratica ai server intermediari

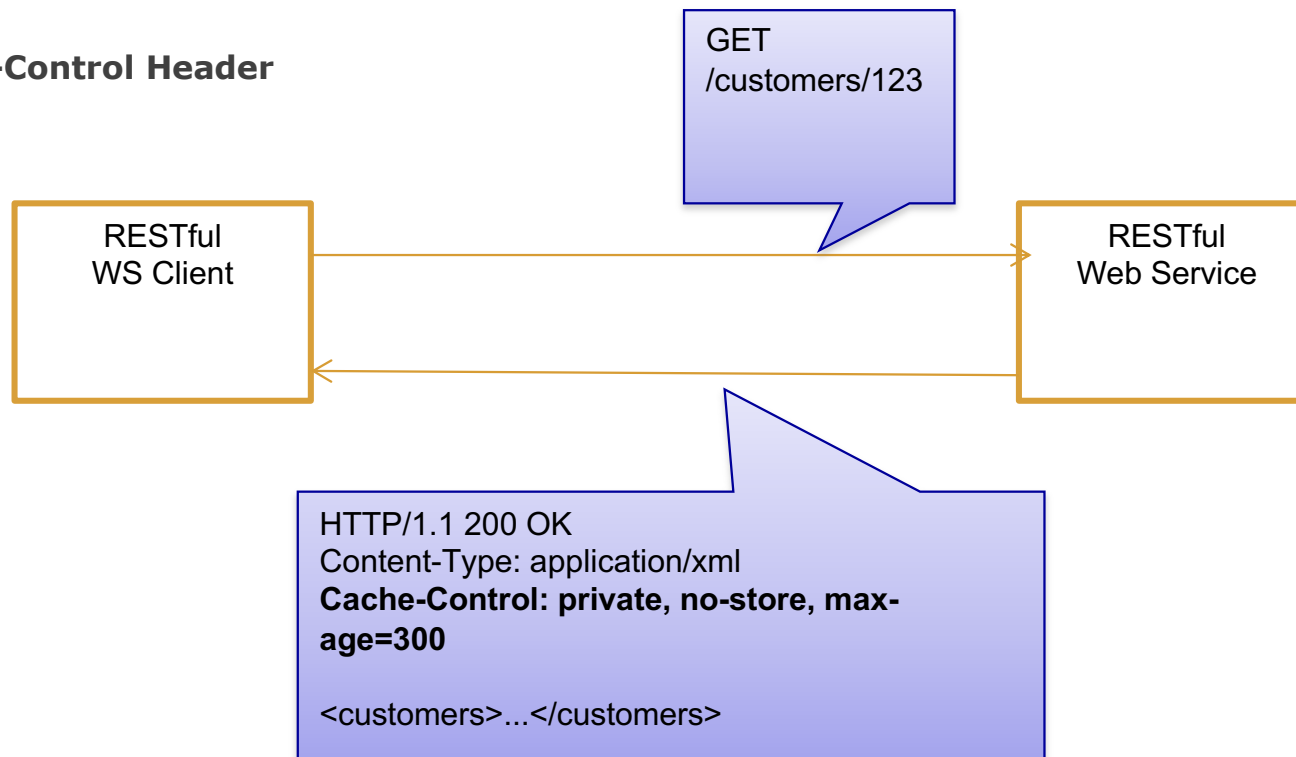
Caching delle risorse: HTTP Caching

Cache-Control Header Directives

- **max-age**: sta ad indicare per quanto a lungo (in secondi) la copia in cache è valida. In caso vi sia anche l'header **Expires**, questa direttiva nel **Cache-Control** ha la precedenza
- **s-maxage**: il significato è identico a max-age, ma è diretta ai server intermediare (proxy/cdn)

Caching delle risorse: HTTP Caching

Cache-Control Header



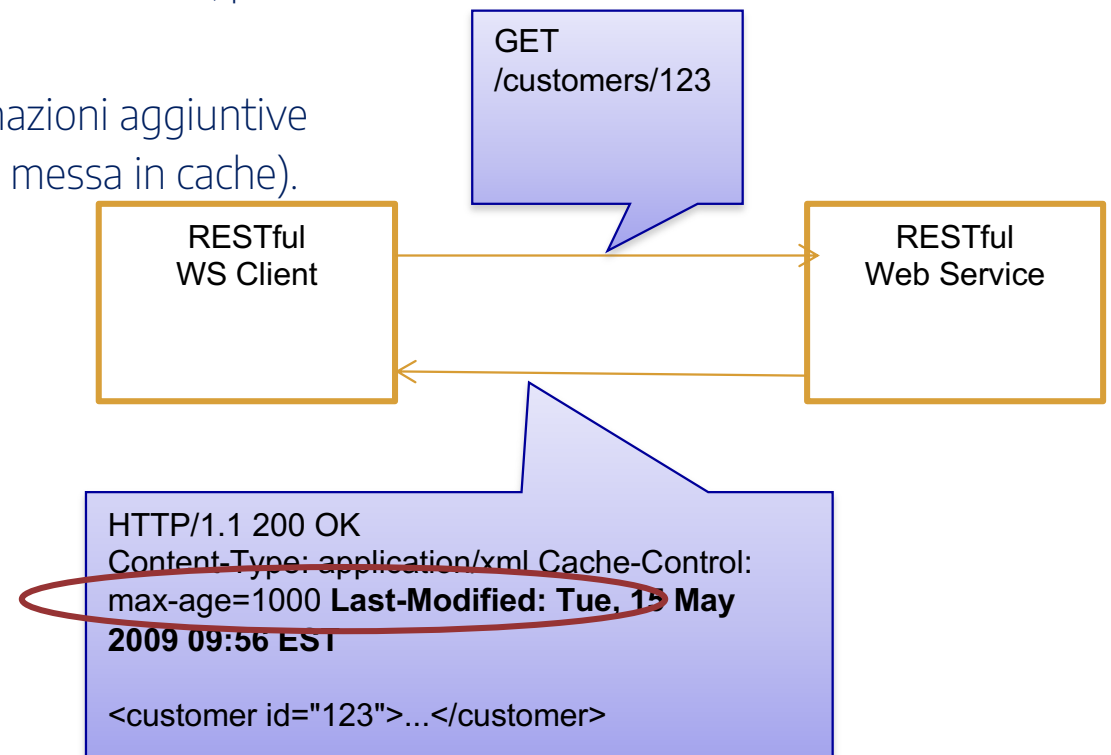
Caching delle risorse: HTTP Caching

Cache Revalidation

Un client che possiede una copia in cache oramai scaduta, può chiedere al server la rivalidazione.

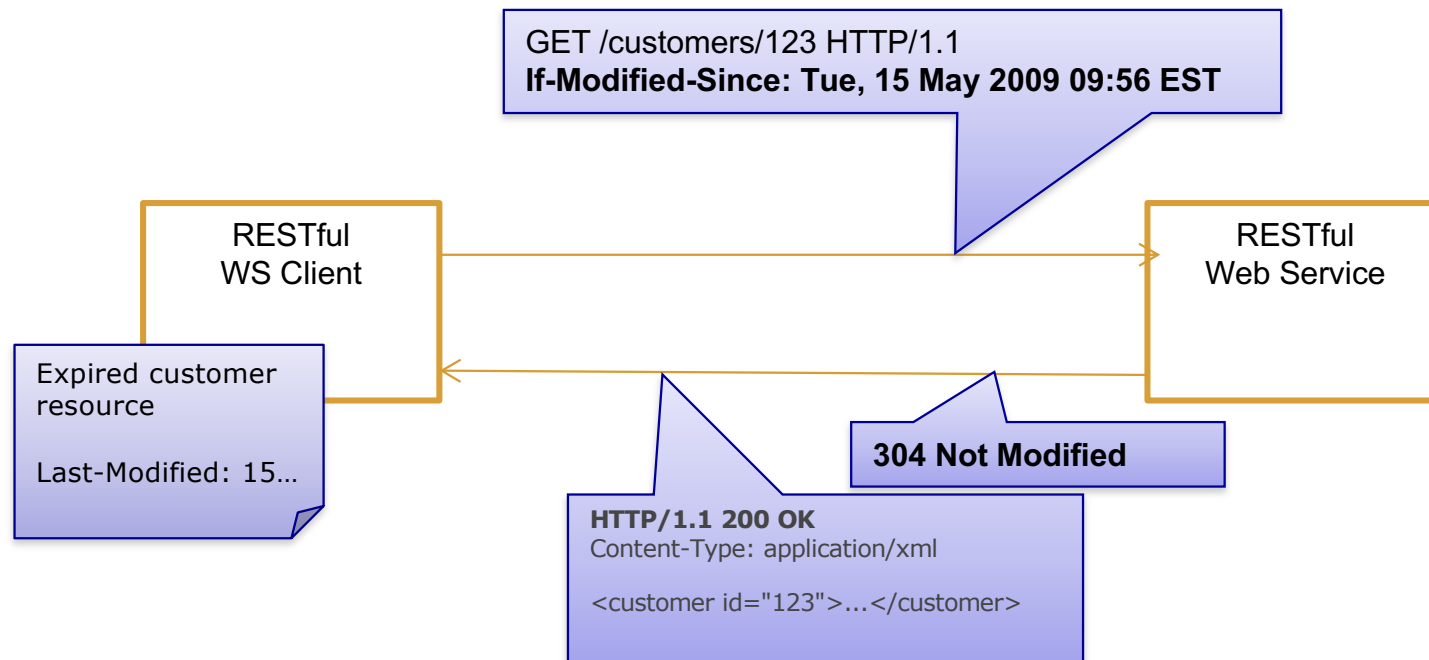
Per farlo però, il server gli deve inviare informazioni aggiuntive assieme alla prima response (quella che verrà messa in cache).

Rappresenta un timestamp
della response inviata
dal server



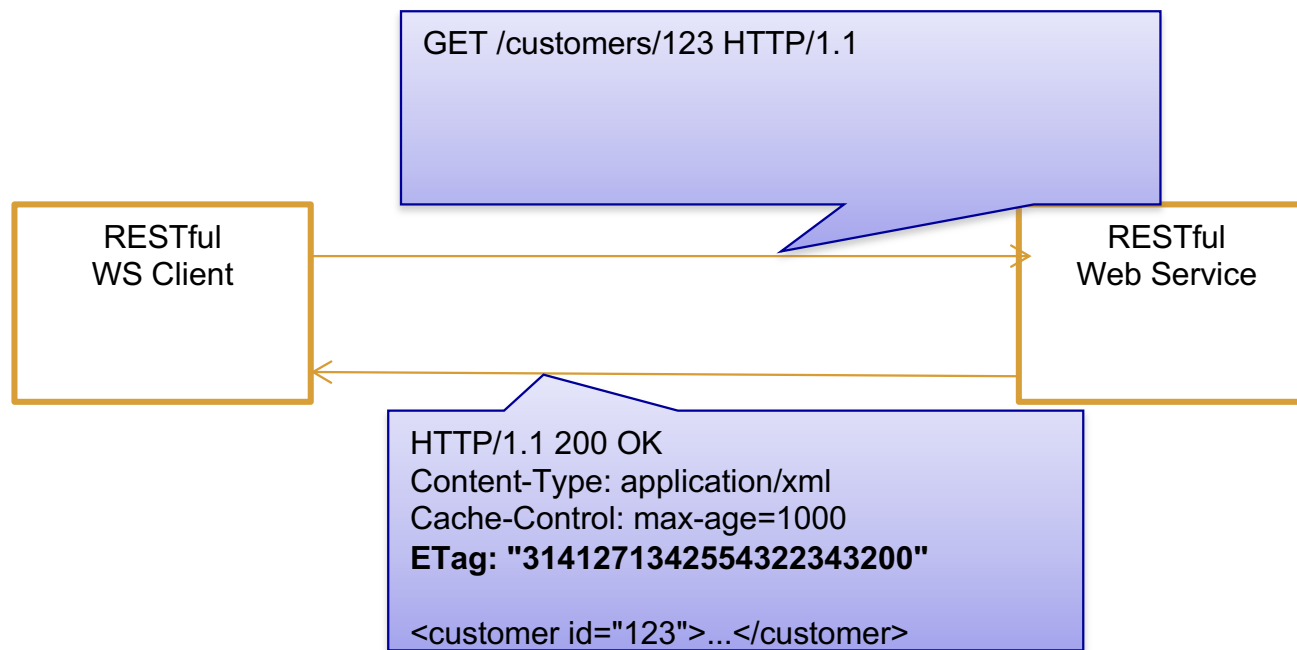
Caching delle risorse: HTTP Caching

Cache Revalidation (If-Modified-Since)



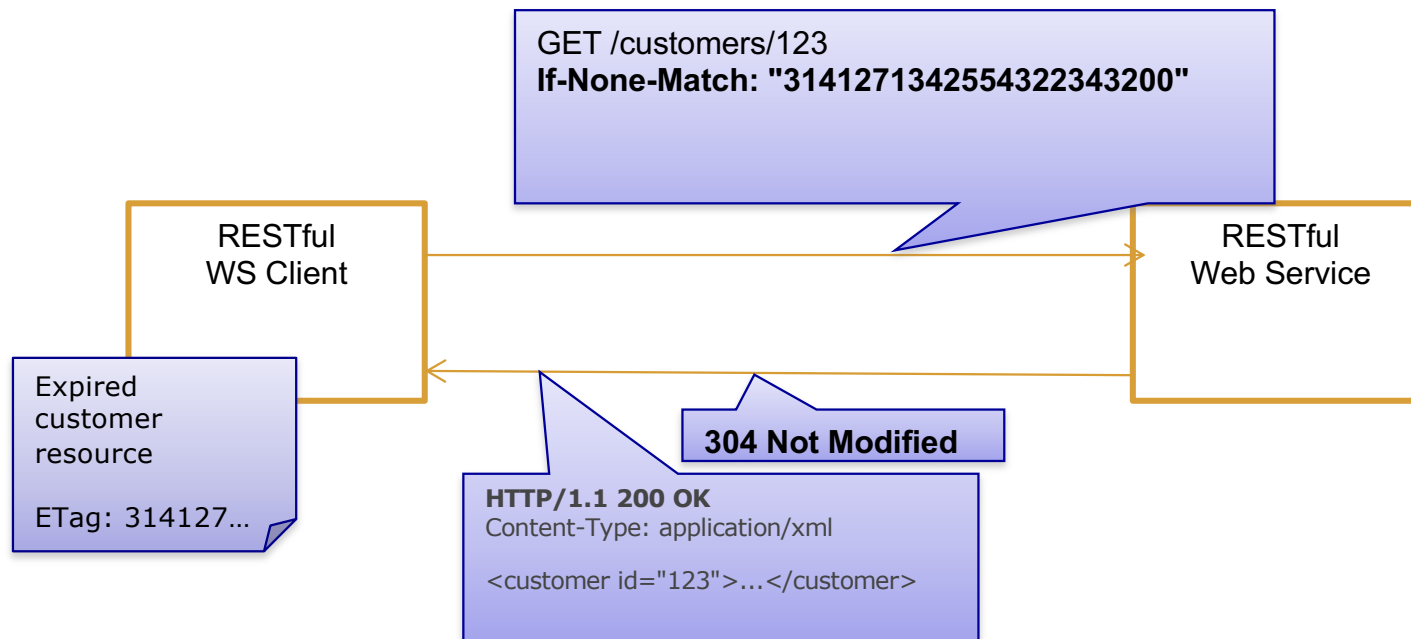
Caching delle risorse: HTTP caching

Cache Revalidation (ETag)



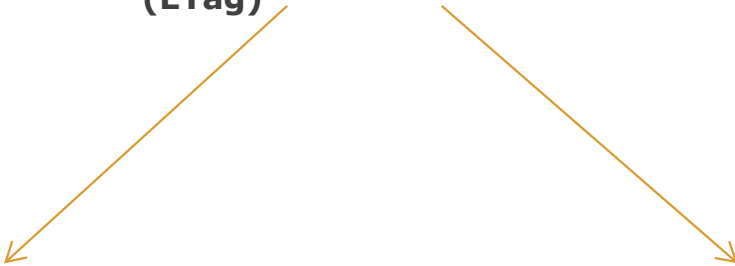
Caching delle risorse: HTTP caching

Cache Revalidation (ETag)



Caching delle risorse: HTTP caching

Cache Revalidation (ETag)



```
graph TD; A[Cache Revalidation (ETag)] --> B[Strong]; A --> C[Weak];
```

Strong

HTTP/1.1 200 OK
Content-Type: application/xml
Cache-Control: max-age=1000
ETag: "3141271342554322343200"

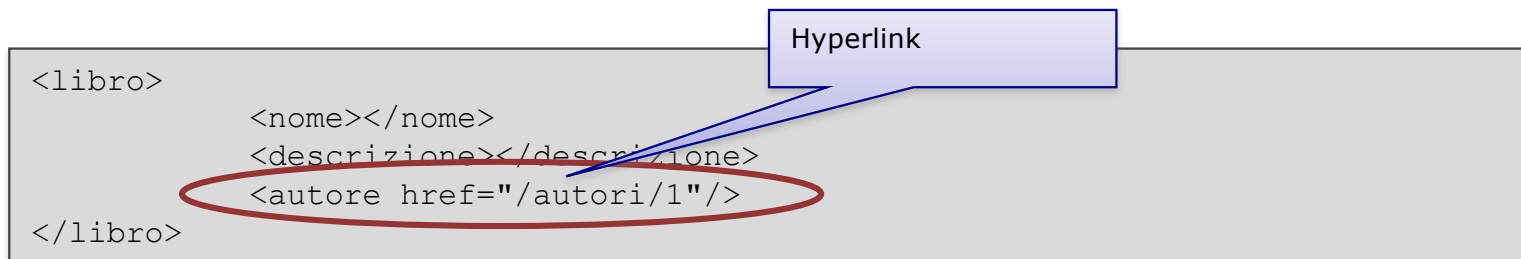
Weak

HTTP/1.1 200 OK
Content-Type: application/xml
Cache-Control: max-age=1000
ETag: **W**/"3141271342554322343200"

Hypermedia as the engine of application state (aka HATEOAS)

Il principio **HATEOAS** intende incoraggiare non solo l'uso di collegamenti per rappresentare risorse composte, ma anche per definire qualsiasi altra relazione tra le risorse e per controllare le transizioni ammissibili tra uno stato e l'altro dell'applicazione.

Per fare un esempio, consideriamo la seguente rappresentazione di un libro nella nostra ipotetica biblioteca:



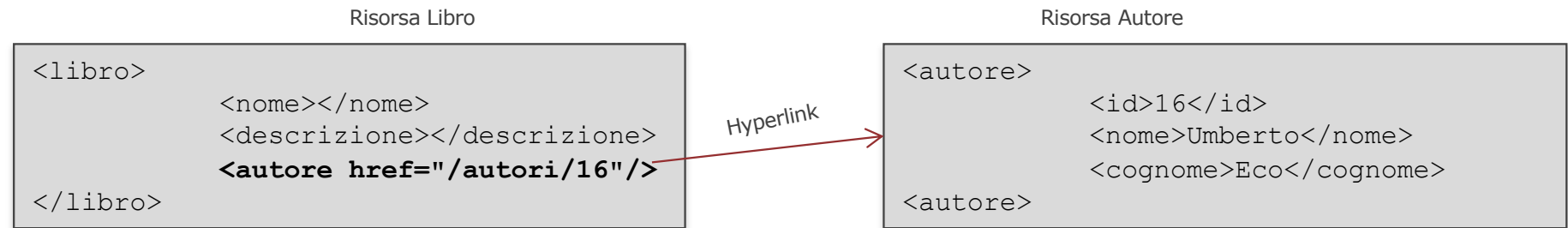
The diagram shows an XML snippet for a book. The snippet is enclosed in a light gray box. Inside the box, the XML code is as follows:

```
<libro>  
  <nome></nome>  
  <descrizione></descrizione>  
  <autore href="/autori/1"/>  
</libro>
```

The line `<autore href="/autori/1"/>` is circled in red. A blue callout box labeled "Hyperlink" points to this line.

Hypermedia as the engine of application state (aka HATEOAS)

Nella **visione REST**, l'esecuzione di un'applicazione può essere rappresentata da una rete di risorse in cui un client naviga seguendo i collegamenti ammissibili tra una risorsa e l'altra. Questa visione probabilmente ci suonerà abbastanza familiare, dal momento che la mettiamo in pratica tutti i giorni navigando tra le pagine del Web.



Sicurezza

Uno degli aspetti cruciali di qualsiasi applicazione Web è la **gestione della sicurezza**. L'esposizione di risorse accessibili tramite i meccanismi tipici del Web, come avviene con l'approccio REST per la creazione di Web Service, non si sottrae a questa necessità.

In linea di massima la sicurezza di un sistema coinvolge almeno i seguenti aspetti:

- **l'autenticazione**, cioè la capacità di identificare le parti coinvolte in una comunicazione
- **l'autorizzazione**, cioè la concessione dei diritti di accesso ad una risorsa in base all'identità
- **la fiducia**, cioè la capacità di confidare nel risultato di un'operazione, come ad esempio l'autenticazione o l'autorizzazione, eseguita da terze parti
- **la riservatezza**, cioè la capacità di mantenere l'informazione privata durante la trasmissione o la memorizzazione
- **l'integrità**, cioè la capacità di prevenire che l'informazione possa essere modificata da terzi

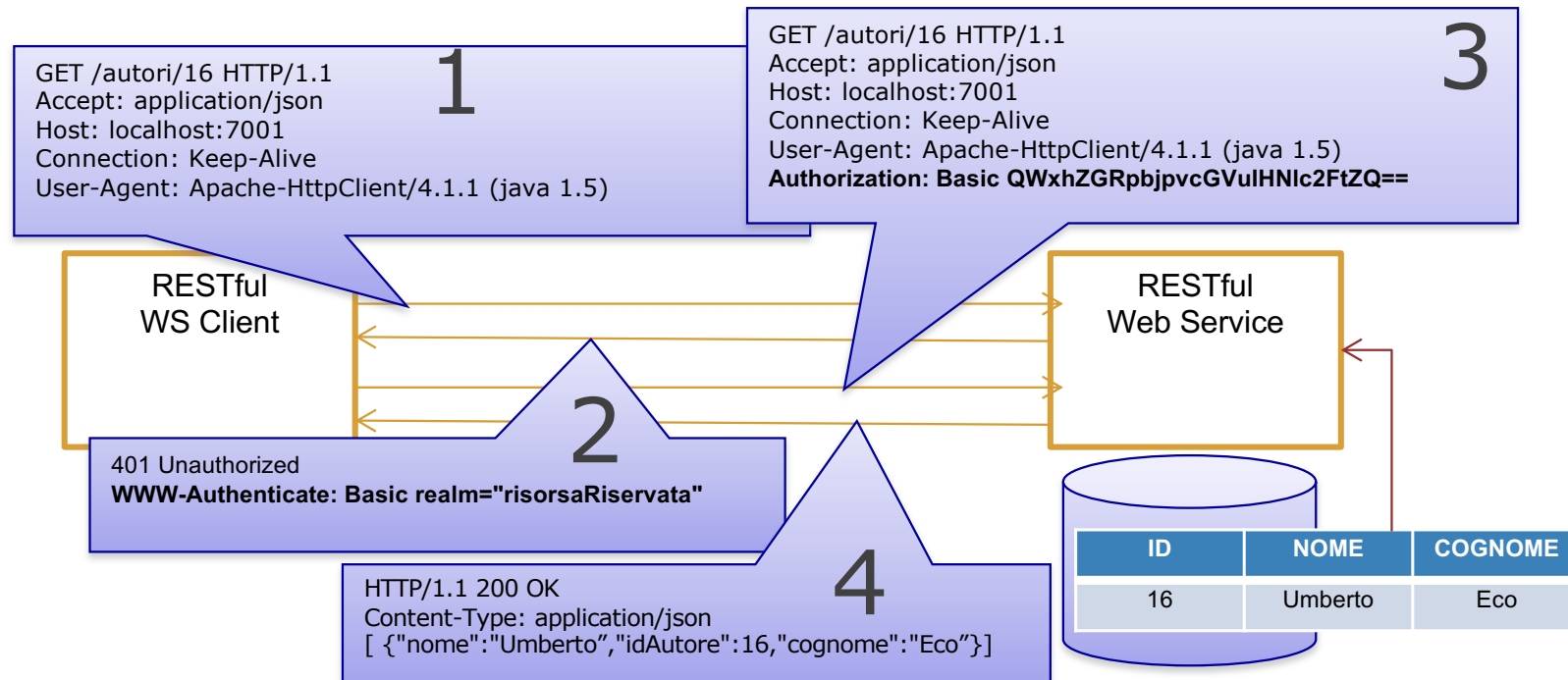
Sicurezza

Fortunatamente questi aspetti sono stati efficacemente affrontati in ambito Web ormai da diverso tempo per poter affermare di essere giunti ad un modello abbastanza maturo.

In effetti, dal momento che l'approccio REST nella creazione di Web Service è largamente basato sul protocollo HTTP, la gestione della sicurezza ne eredita le caratteristiche ormai collaudate.

In sostanza, per gestire l'identità nell'ambito di un Web Service RESTful possono essere utilizzati i meccanismi propri del protocollo HTTP, come ad esempio l'**HTTP Basic Authentication** o l'**HTTP Digest Authentication**.

Autenticazione > Basic Access Authentication



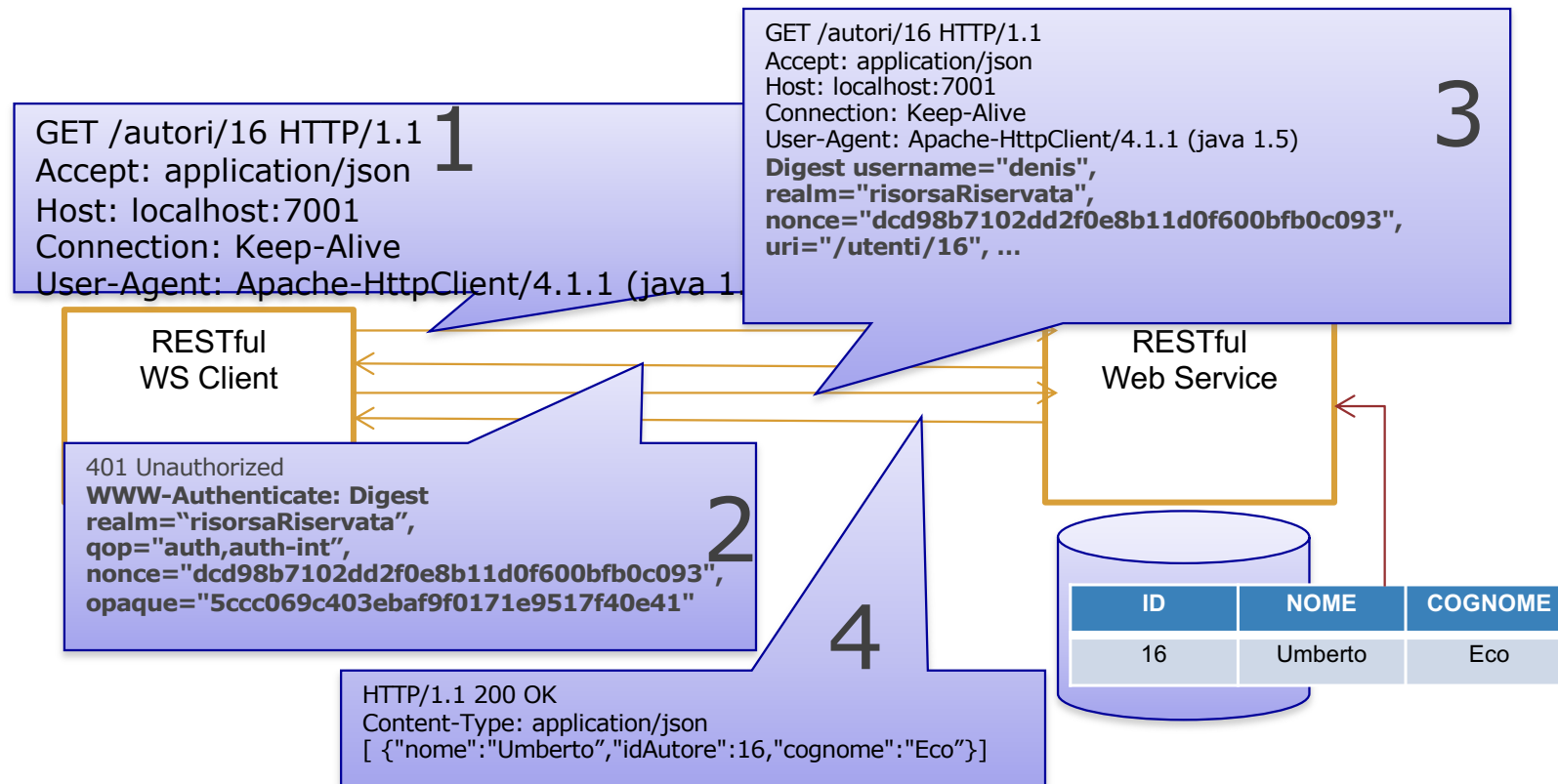
Autenticazione > Basic Access Authentication

La password è inviata codificata con l'algoritmo Base64, che è reversibile.

Un utente che dovesse venire a conoscenza della password codificata potrebbe risalire a quella in chiaro.

Per questo motivo è necessario mettere in sicurezza il canale di trasporto, mediante l'utilizzo del protocollo "sicuro" HTTPS

Autenticazione > Digest Access Authentication



Autenticazione > Digest Access Authentication

E' un meccanismo più sicuro di Basic Authentication.

Tecnicamente, utilizza l'algoritmo di hashing MD5 + un nonce (numero casuale) per evitare attacchi di tipo replay attack

Pur un metodo di autenticazione più sicuro di Basic Access Authentication, l'utilizzo di protocolli sicuri HTTPS è sempre consigliato (cifatura del traffico)

2021-2023 Cloud specialist

UF: Microservices

Docente: Denis Maggiorotto

API aggregation with GraphQL



REST vs GraphQL

Le API REST tradizionali funzionano con il concetto di «risorse» che il server gestisce. Questo le rende lo strumento ideale finché la struttura restituita dalle API equivale alla rappresentazione della risorsa gestita lato server (API interface = resource representation).

Esistono casi in cui l'interfaccia esposta da una REST API si discosta dalla rappresentazione della risorsa (API interface != resource representation). In questi casi il modello REST è meno efficiente.

REST vs GraphQL

Inoltre, spesso il client ha bisogno di dati da più risorse contemporaneamente. Ad esempio, richiedere il post di un utente su di un blog ed i relativi followers. In genere questo viene risolto facendo in modo che il client effettui più richieste o facendo in modo che il server fornisca dati aggiuntivi che potrebbero non essere sempre richiesti, portando a dimensioni di risposta maggiori.

Overfetching: ottenere in risposta più dati del necessario

Underfetching: un'API non restituisce dati a sufficienza e costringe il client ad effettuare più richieste (n+1 requests problem)

Le API REST spesso si mappano 1 ad 1 con il tipo di client (UI) che le consuma. Quando il client cambia occorre modificare le API

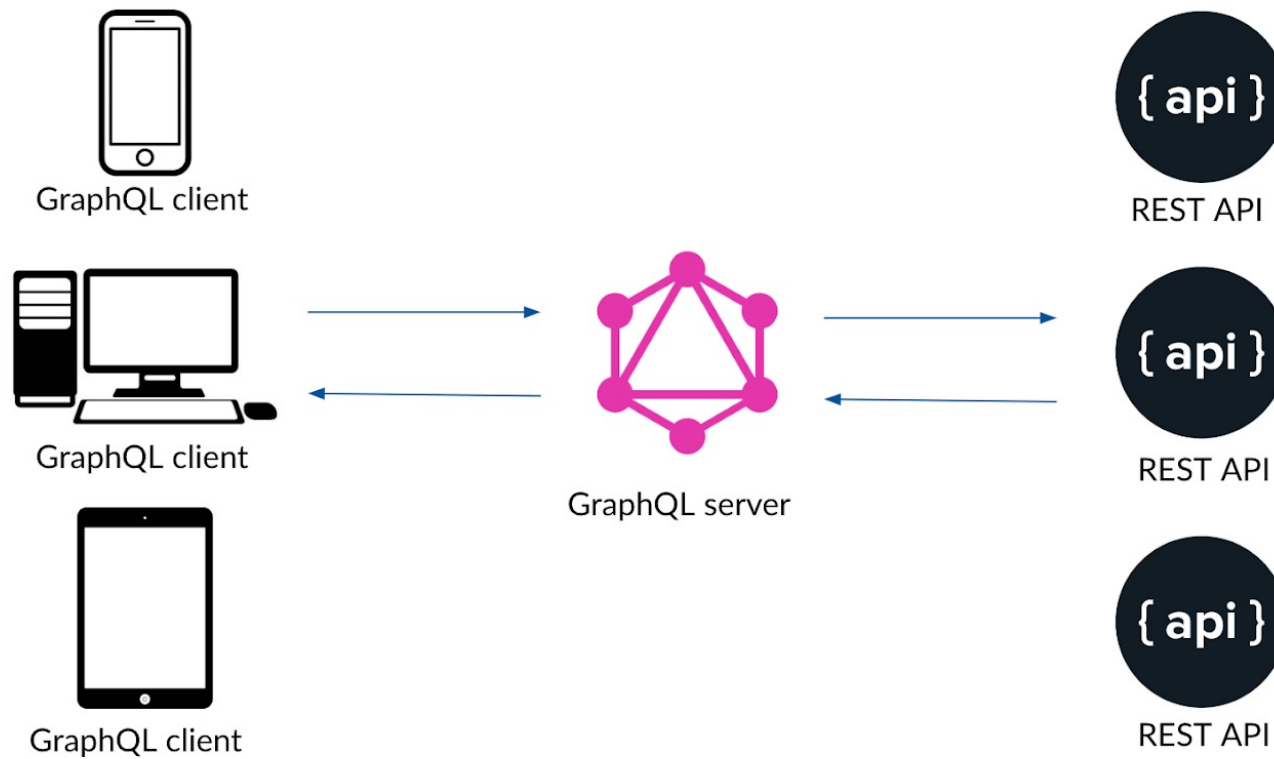
REST vs GraphQL



REST vs GraphQL

GraphQL offre una soluzione ad entrambi requisiti espressi nella slide precedente. Consente al client di specificare esattamente quali dati sono desiderati, inclusa la navigazione di risorse figlie (consente quindi di effettuare più query in una singola richiesta).

REST vs GraphQL



I vantaggi di GraphQL

Mai più overfetching: i client ottengono esattamente i dati richiesti

Mai più undefetching: i client ottengono esattamente i dati richiesti

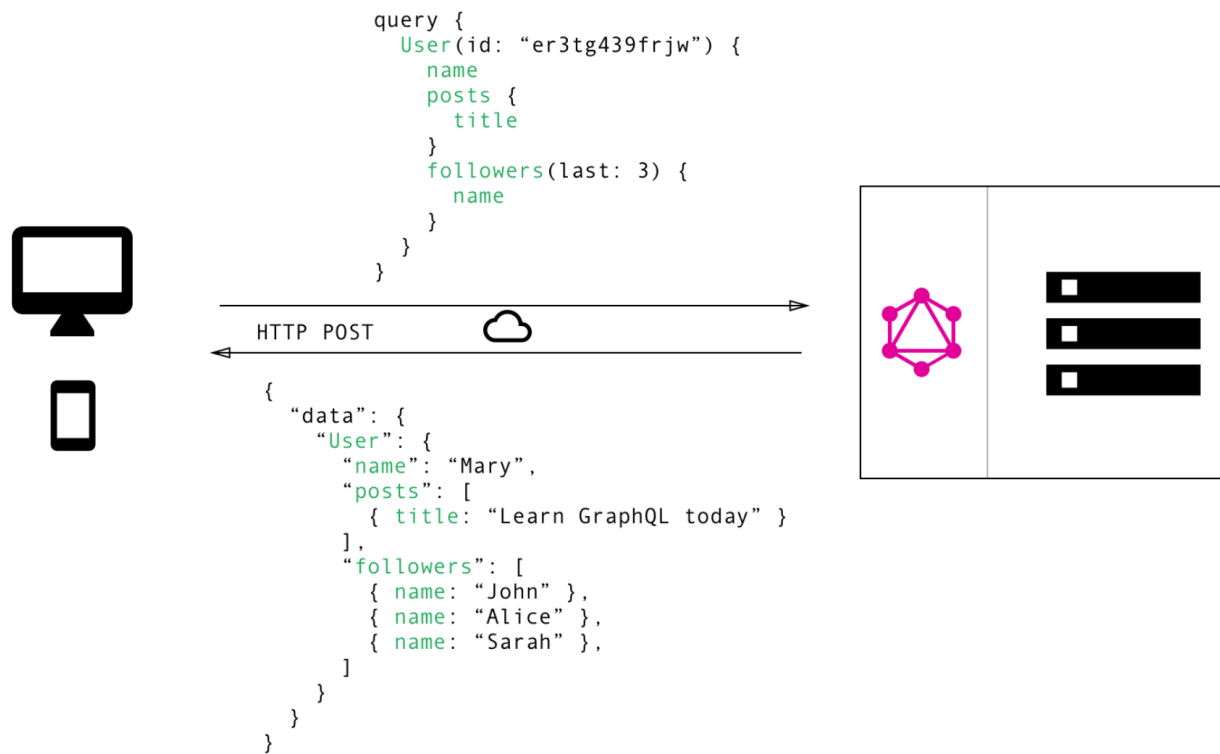
Iterazioni di sviluppo dei prodotti più rapide: non è necessario modificare le API lato server quando il client (spesso la UI) cambia

Accounting di quali dati sono richiesti ai client (possibile grazie al fatto che i client richiedono espressamente i dati in risposta). Queste informazioni sono utili per deprecare funzionalità delle API non più utilizzate

I vantaggi di GraphQL

Type-safe query language: grazie all'utilizzo di schema, le API descrivono le proprie funzionalità in modo type-safe

GraphQL is the better REST



GraphQL: Schema Definition Language (SDL)

GraphQL ha il proprio sistema di tipi che viene utilizzato per definire lo schema di un'API. La sintassi per scrivere gli schemi si chiama Schema Definition Language (SDL).

```
type Person {  
  name: String!  
  age: Int!  
}
```

```
type Post {  
  title: String!  
  author: Person!  
}
```

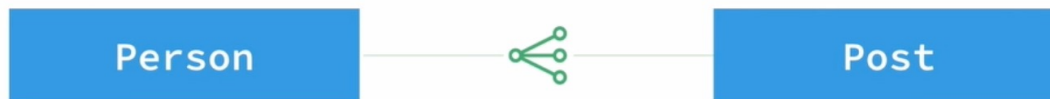
! significa che il campo è obbligatorio

GraphQL: Relations

E' possibile specificare una relazione tra le entità. Nell'esempio che segue il campo **posts** in Person è un riferimento ai post scritti da un utente (one-to-many relation)

```
type Person {  
  name: String!  
  age: Int!  
  posts: [Post!]!  
}
```

```
type Post {  
  title: String!  
  author: Person!  
}
```



GraphQL: Query

Rispetto al paradigma REST, GraphQL prevede l'esposizione di un singolo URL per interrogare le risorse (i dati che il client vuole ottenere sono passati sul body della richiesta).



GraphQL: Query

Il client può in alcuni casi selezionare più dati che gli verranno restituiti in risposta (es: campo **age** oltre a **name**)



GraphQL: Query

Il client può in alcuni casi richiedere anche risorse «figlie», come nell'esempio che segue i post di un utente.

```
{
  allPersons {
    name
    posts {
      title
    }
  }
}
```

```
{
  "allPersons": [
    {
      "name": "Johnny",
      "posts": [
        { "title": "GraphQL is awesome" },
        { "title": "Relay is a powerful GraphQL Client" }
      ]
    },
    {
      "name": "Sarah",
      "posts": [
        { "title": "How to get started with React & GraphQL" }
      ]
    },
    {
      "name": "Alice",
      "posts": []
    }
  ]
}
```

GraphQL: Mutations

Operazioni che hanno l'obiettivo di alterare le risorse lato server.
Normalmente di tre tipo:

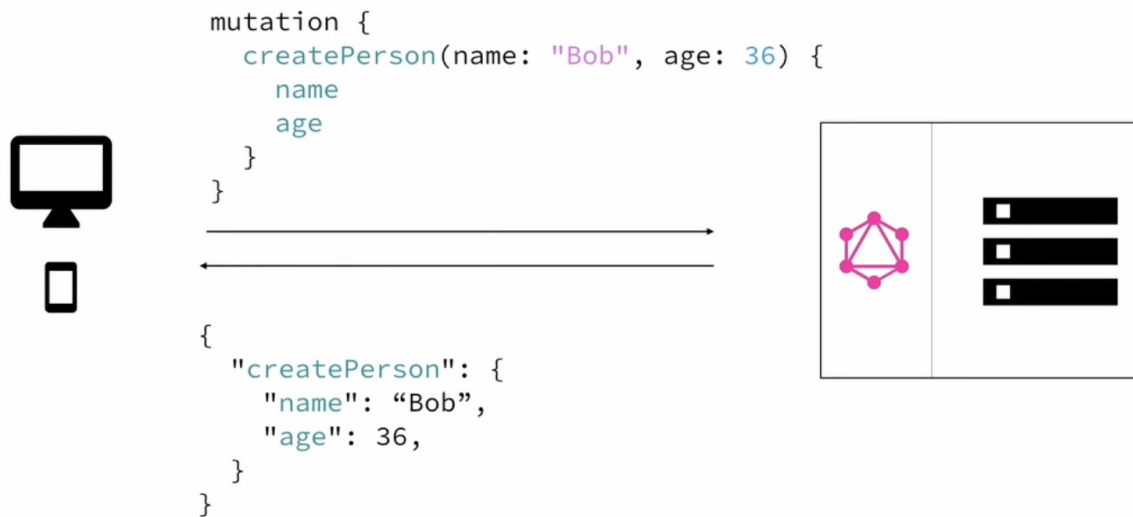
Creazione di nuovi dati

Aggiornamento di dati esistenti

Cancellazione di dati esistenti

GraphQL: Mutations

Sintassi simile alle query ma con **mutation** come root keyword
E' possibile ottenere in output la risorsa che è stata creata/modificata tramite mutation.



GraphQL: Mutations

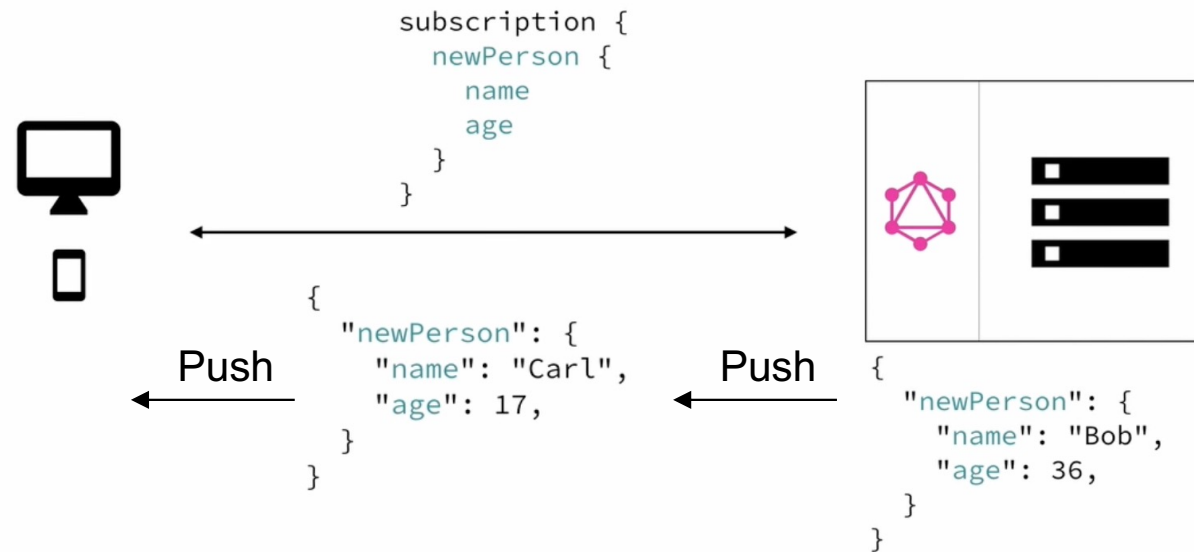
Ad esempio se le entità lato server vengono dotate di un ID univoco, tramite una mutation possiamo ottenerlo in risposta alla creazione.

```
type Person {  
  id: String!  
  name: String!  
  age: Int!  
}
```

```
mutation {  
  createPerson(name: "Alice", age: 36) {  
    id  
  }  
}
```

GraphQL: Subscriptions

Con una subscription, un client si sottoscrive agli eventi di creazione/modifica/cancellazione di un'entità lato server.



GraphQL: Schema

Definiscono le funzionalità di un'API (es: quali entità il server fornisce, quali dati il client può richiedere per ciascuna)

Rappresentano un contratto tra client e server

Sono una collezione di tipi GraphQL con una radice comune (root type)

GraphQL: Root types

```
type Query {  
  ...  
}  
  
type Mutation {  
  ...  
}  
  
type Subscription {  
  ...  
}
```

GraphQL: Query type

```
{  
  allPersons {  
    name  
  }  
}  
  
type Query {  
  allPersons(last: Int): [Person!]!  
}
```


GraphQL: Mutation type

```
mutation {  
  createPerson(name: "Bob", age: 36) {  
    id  
  }  
}
```

```
type Mutation {  
  createPerson(name: String!, age: String!): Person!  
}
```

GraphQL: Subscription type

```
subscription {  
  newPerson {  
    name  
    age  
  }  
}  
  
type Subscription {  
  newPerson: Person!  
}
```

GraphQL: Person type full schema

```
type Query {  
  allPersons(last: Int!): [Person!]!  
  allPosts(last: Int!): [Post!]!  
}  
  
type Mutation {  
  createPerson(name: String!, age: String!): Person!  
  updatePerson(id: ID!, name: String!, age: String!): Person!  
  deletePerson(id: ID!): Person!  
}  
  
type Subscription {  
  newPerson: Person!  
}
```

```
type Person {  
  id: ID!  
  name: String!  
  age: Int!  
  posts: [Post!]!  
}  
  
type Post {  
  title: String!  
  author: Person!  
}
```

GraphQL: Full schema

```
type Query {
  allPersons(last: Int!): [Person!]!
  allPosts(last: Int!): [Post!]!
}

type Mutation {
  createPerson(name: String!, age: String!): Person!
  updatePerson(id: ID!, name: String!, age: String!): Person!
  deletePerson(id: ID!): Person!
  createPost(title: String!): Post!
  updatePost(id: ID!, title: String!): Post!
  deletePost(id: ID!): Post!
}

type Subscription {
  newPerson: Person!
  updatedPerson: Person!
  deletedPerson: Person!
  newPost: Post!
  updatedPost: Post!
  deletedPost: Post!
}

type Person {
  id: ID!
  name: String!
  age: Int!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  author: Person!
}
```

GraphQL architectural uses cases

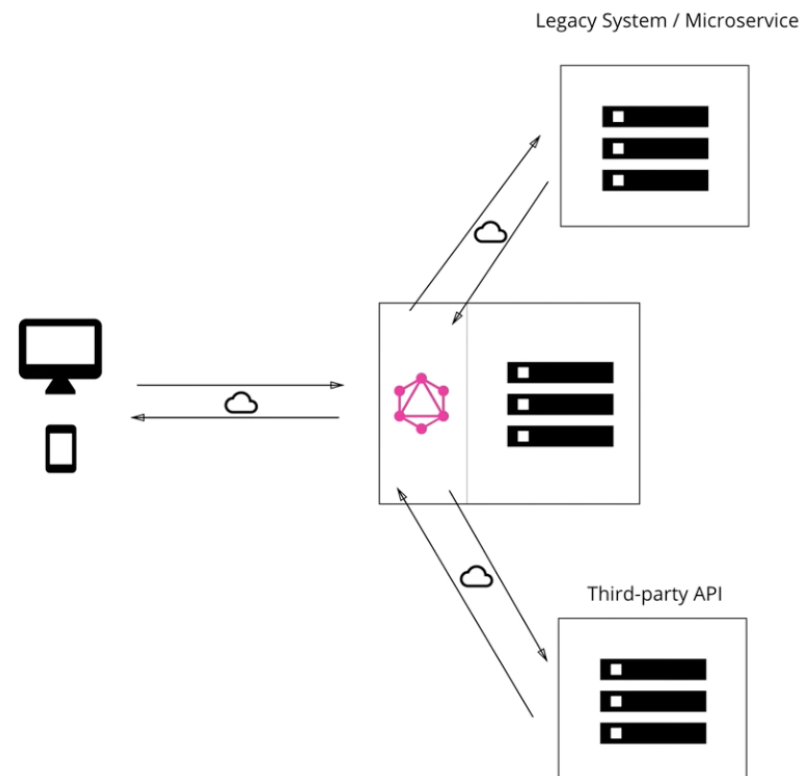
GraphQL server connesso ad un database

GraphQL server per integrare sistemi di terze parti (data enrichment)

GraphQL server per orchestrare microservizi REST nel backend (API aggregation)*

* Ci focalizzeremo su questo use case

GraphQL architectural uses cases



GraphQL: Resolver functions

GraphQL query o mutations consistono in un insieme di campi
Il GraphQL server ha una funzione «resolver» per ogni campo
Lo scopo di ogni resolver è quello di gestire i dati per conto del campo corrispondente

GraphQL: Resolver functions



```
query {  
  User(id: "er3txsa9frju") {  
    name  
    friends(first: 5) {  
      name  
      age  
    }  
  }  
}
```

Diagram showing a client (represented by a monitor and smartphone icon) sending a query to a server (represented by a cloud icon) via a network connection (represented by a double-headed arrow).

Resolvers



```
User(id: String!): User  
name(user: User!): String!  
age(user: User!): Int!  
friends(first: Int, user: User!): [User!]!
```